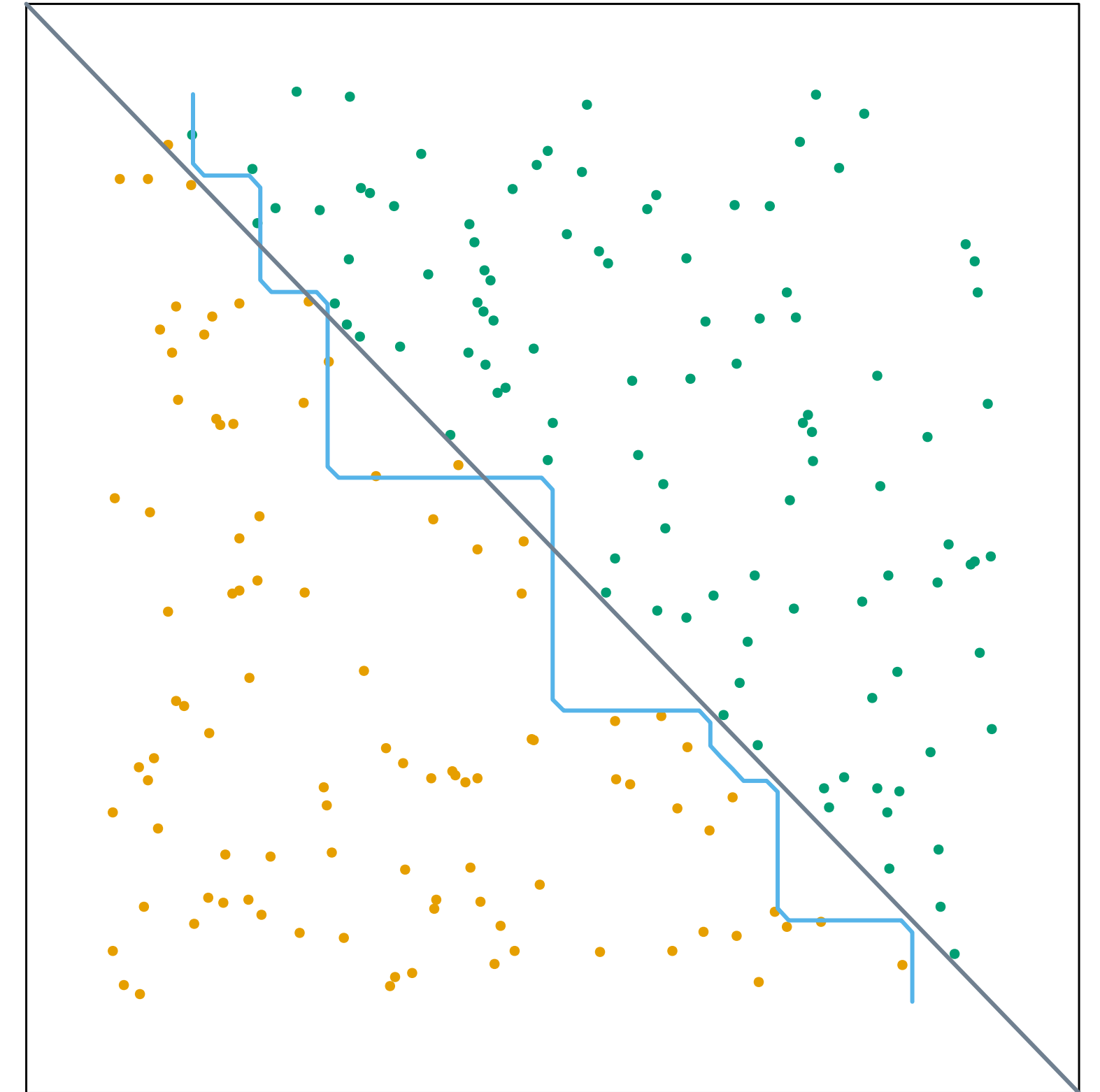
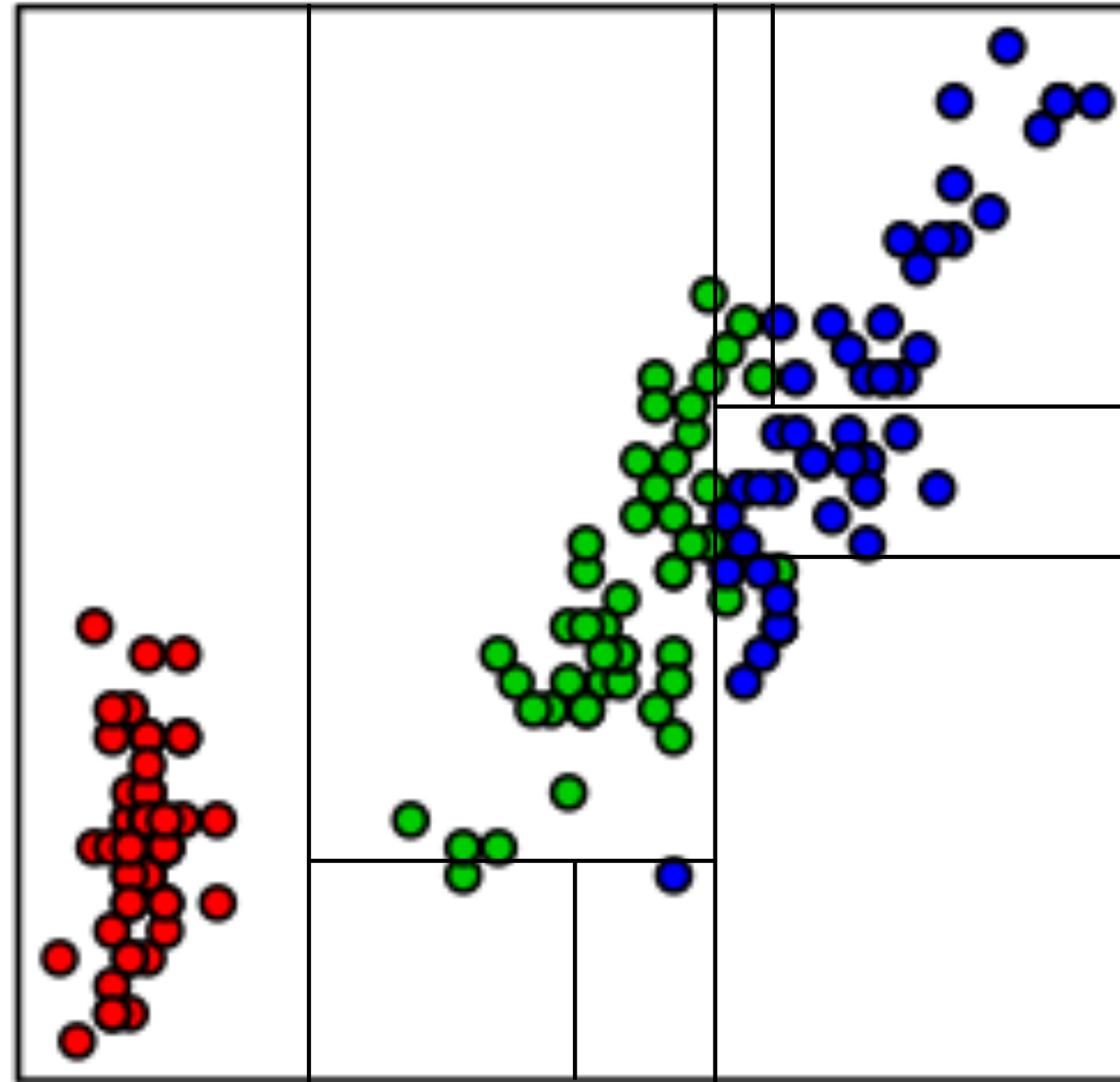


Inductive bias of decision trees



- Divide space into nested regions
- Piecewise constant in regions
- Axis-parallel splits
- Decisions expressed as simple rules: sepal width ≤ 5.2 & sepal length $\in [2.5, 4.0]$

Learning objectives— decision trees

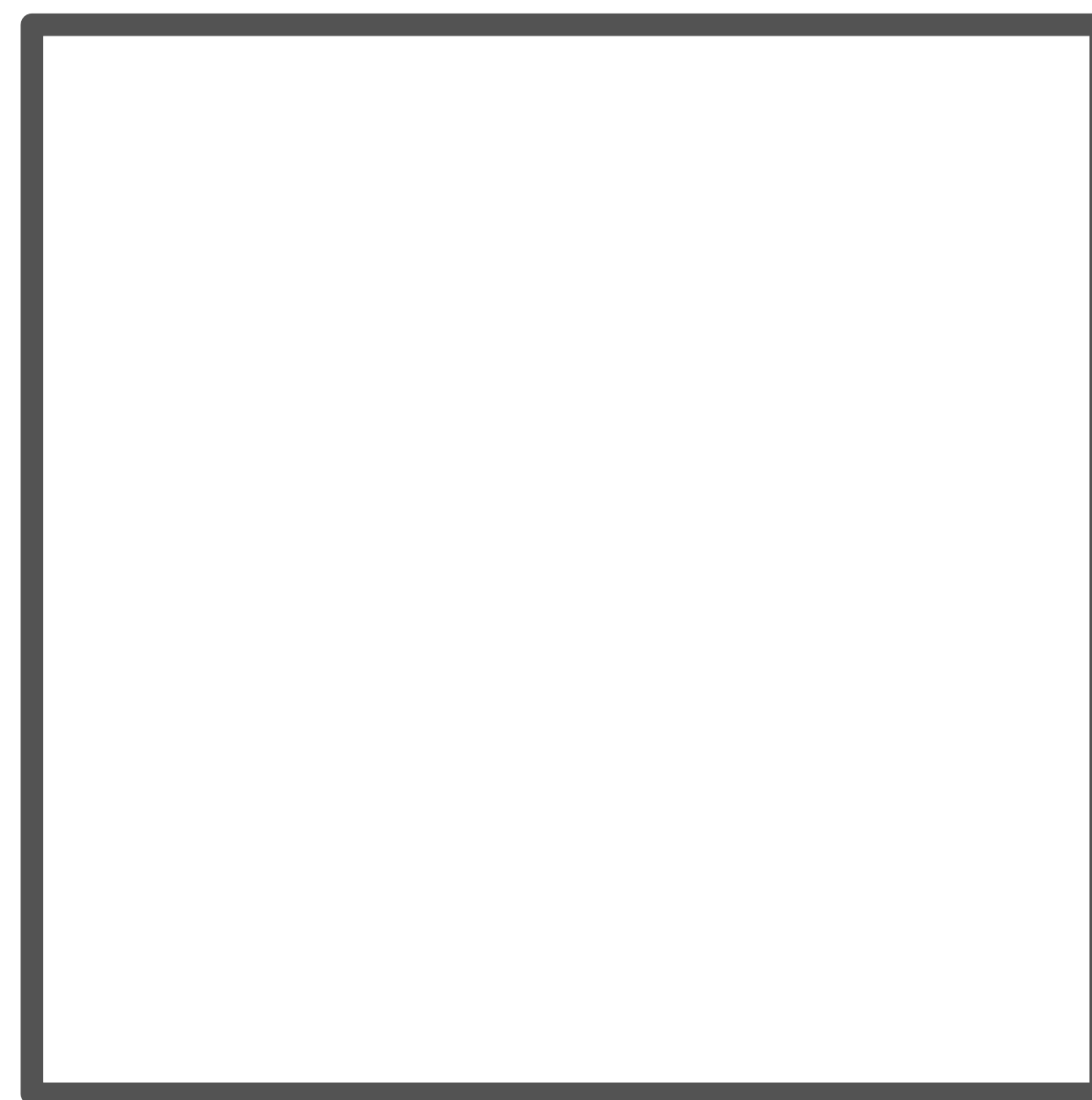
- What is a decision tree? How do we use it to predict?
- What is the inductive bias of a decision tree?
- How can we grow a decision tree?
- How do we use mutual information as a splitting criterion?
- How can we prune a decision tree? Why would we?
- What is a validation set, how does it differ from a test set, and why do we need one?

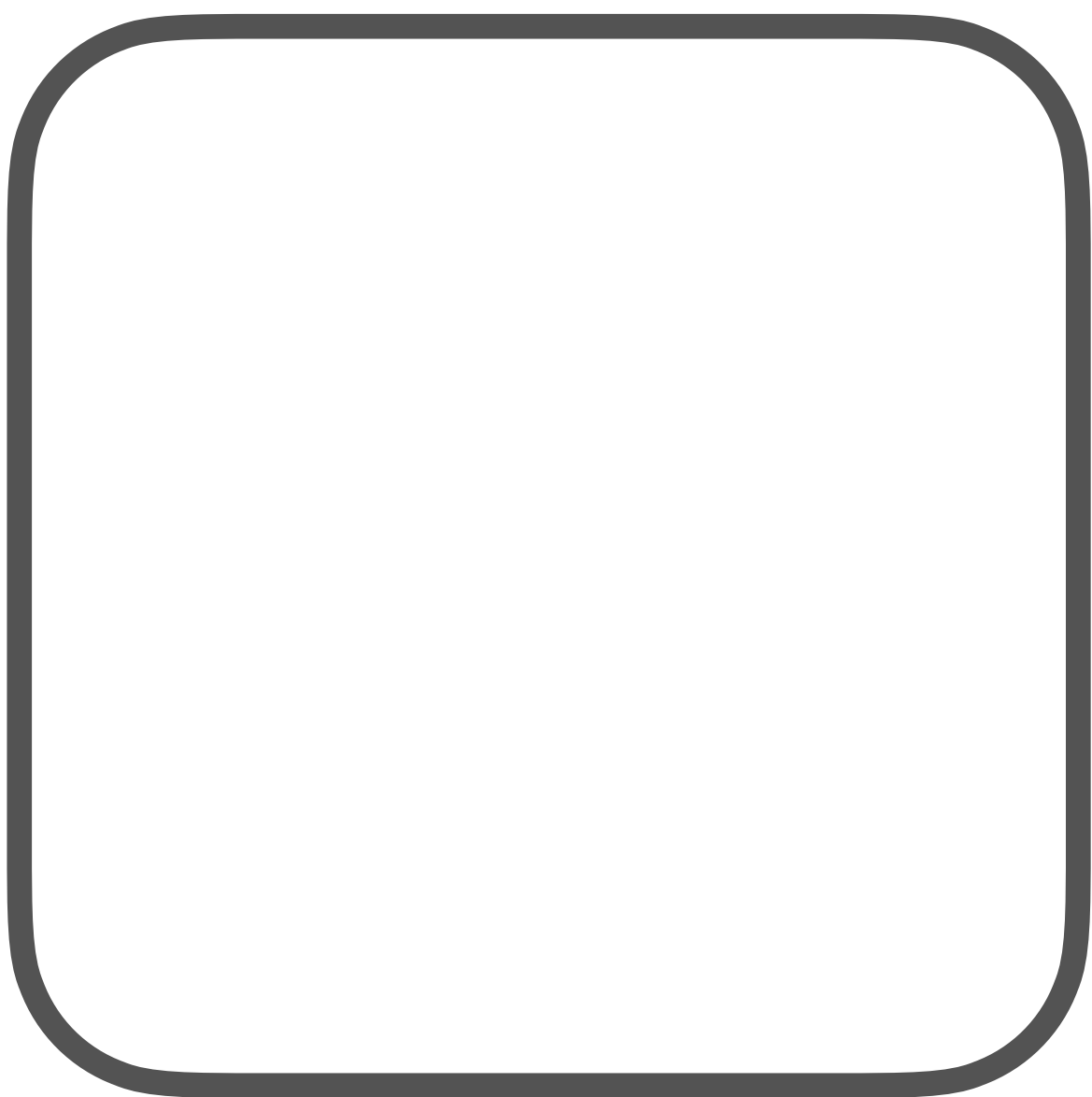
Try it at home!

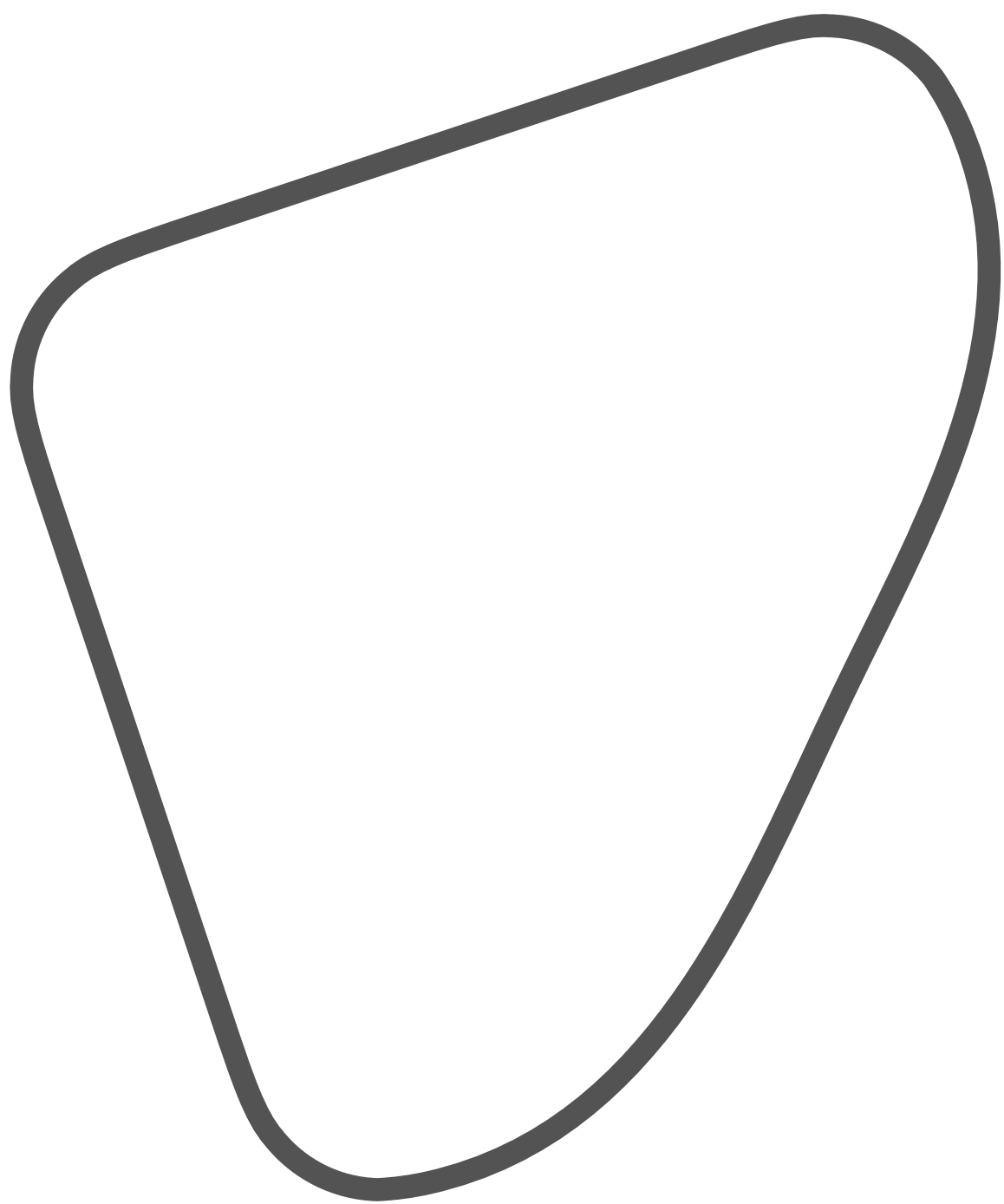


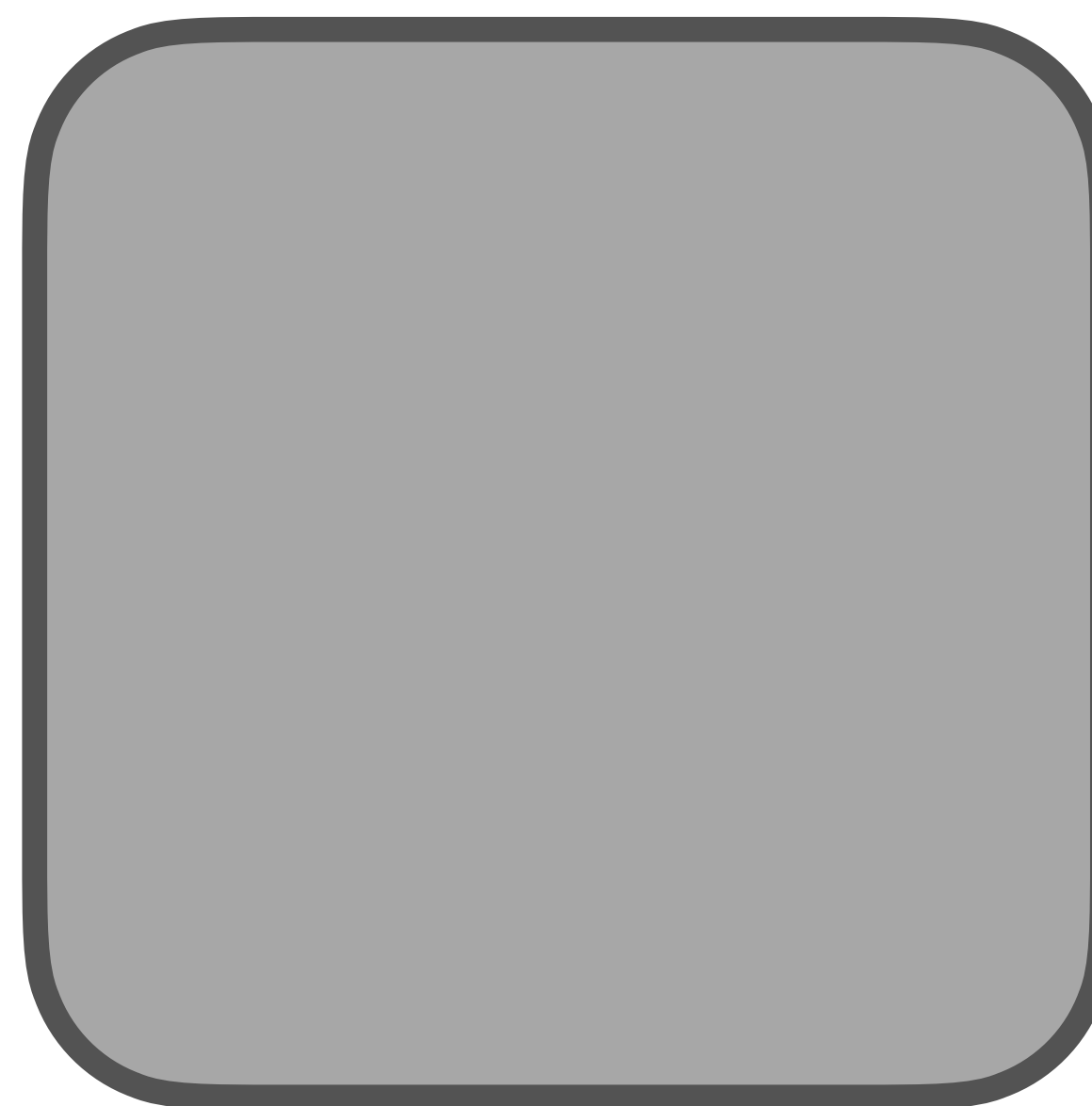
Result

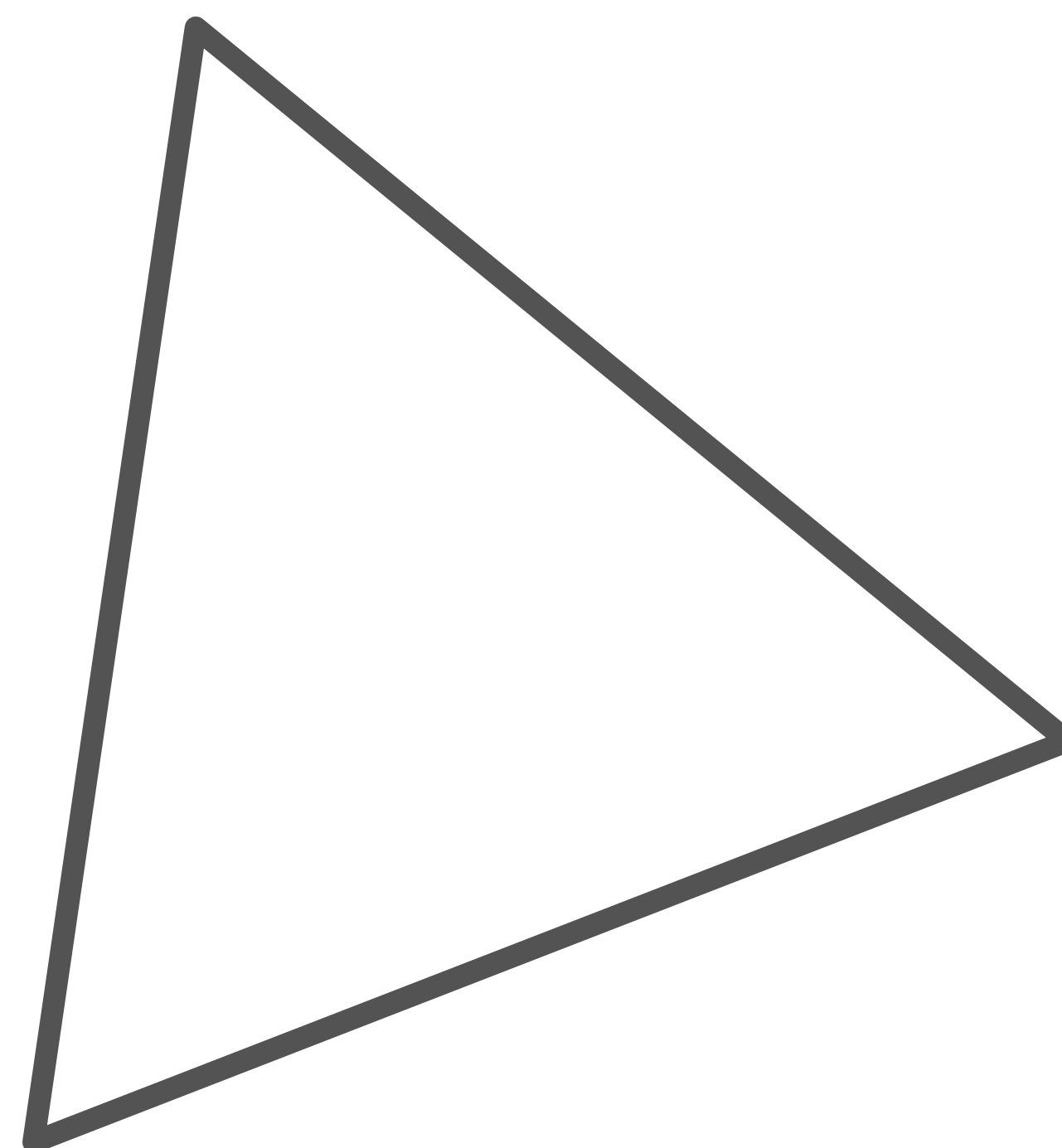
- Features:
- Learned tree:

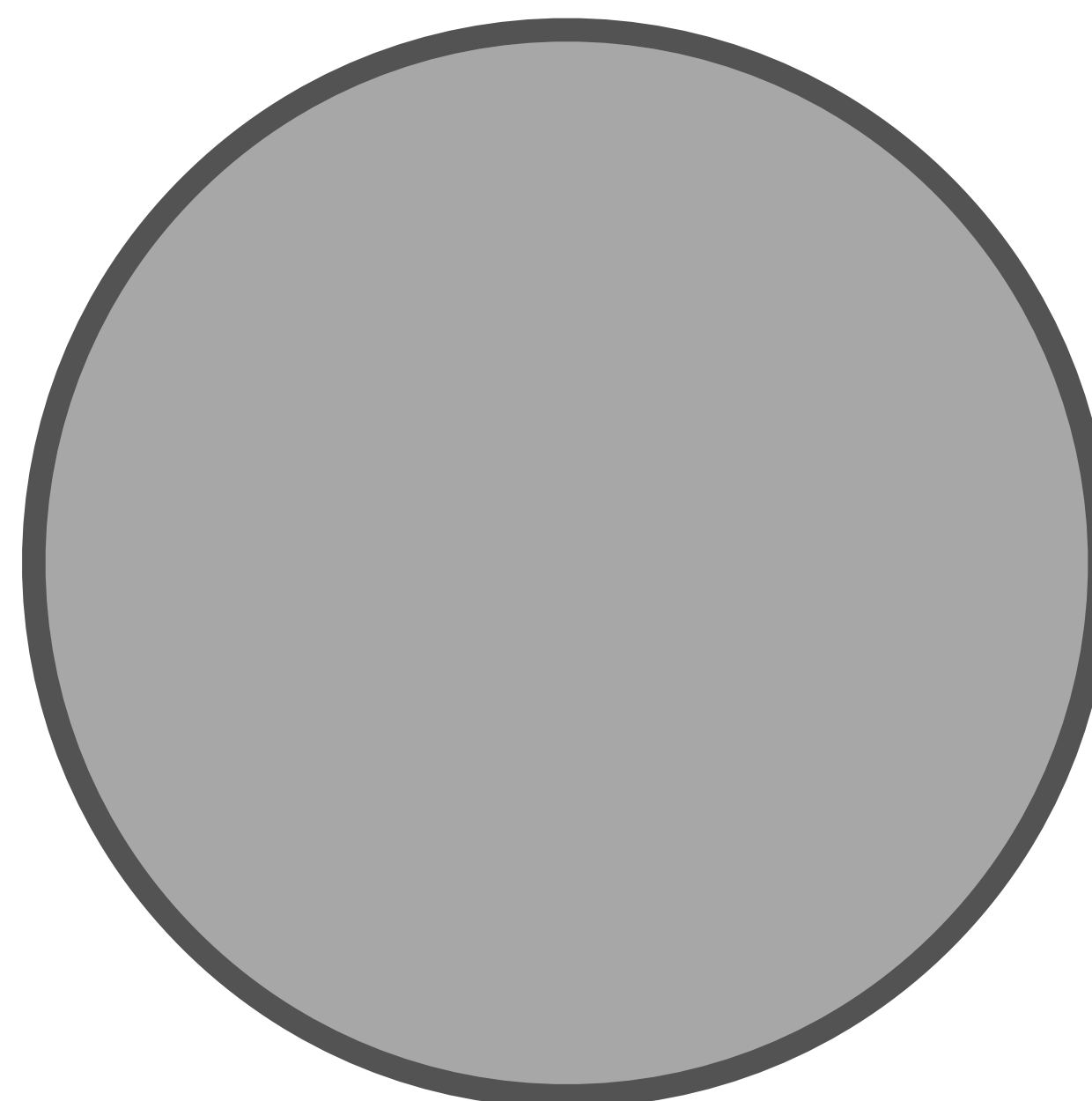


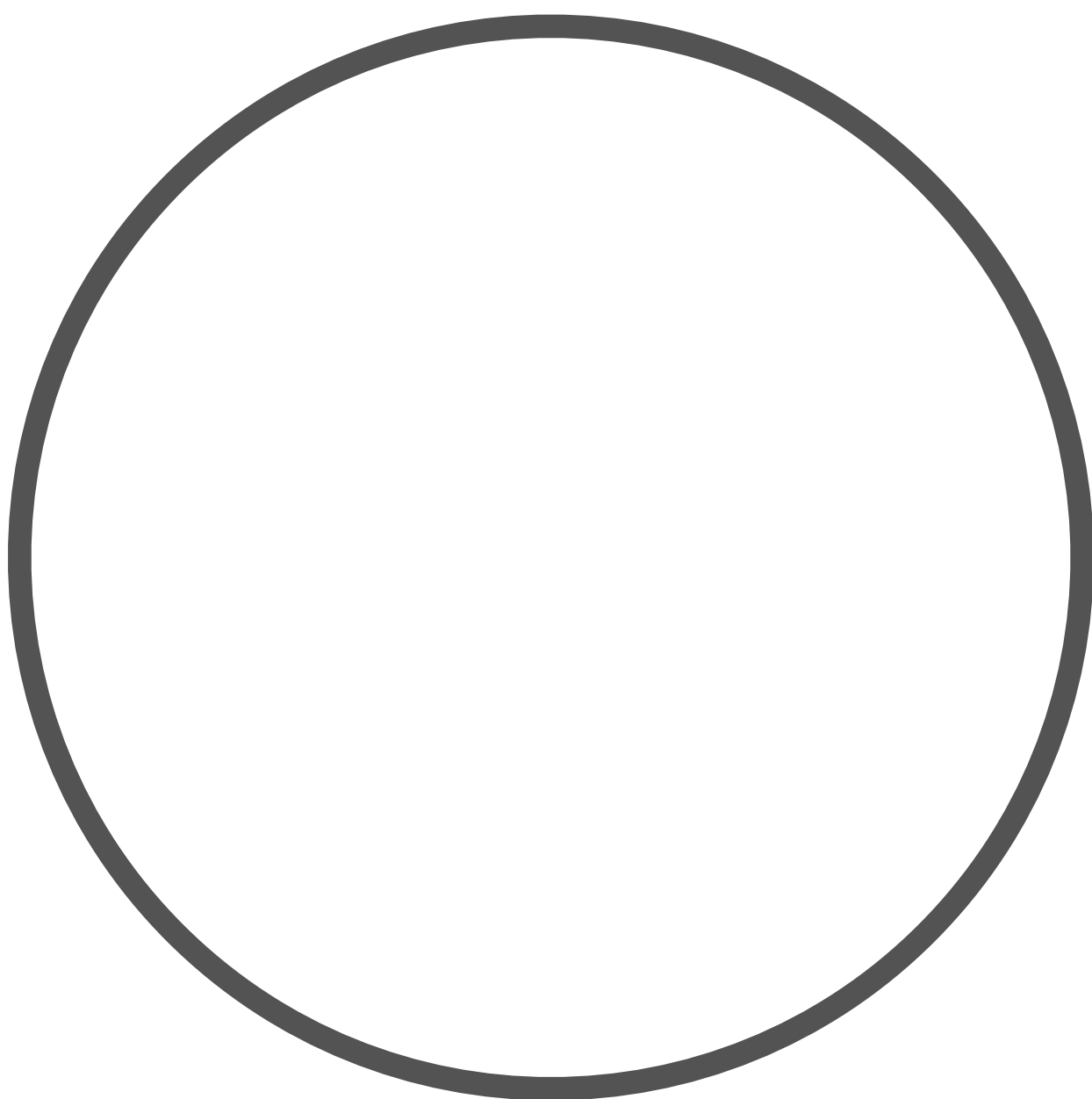












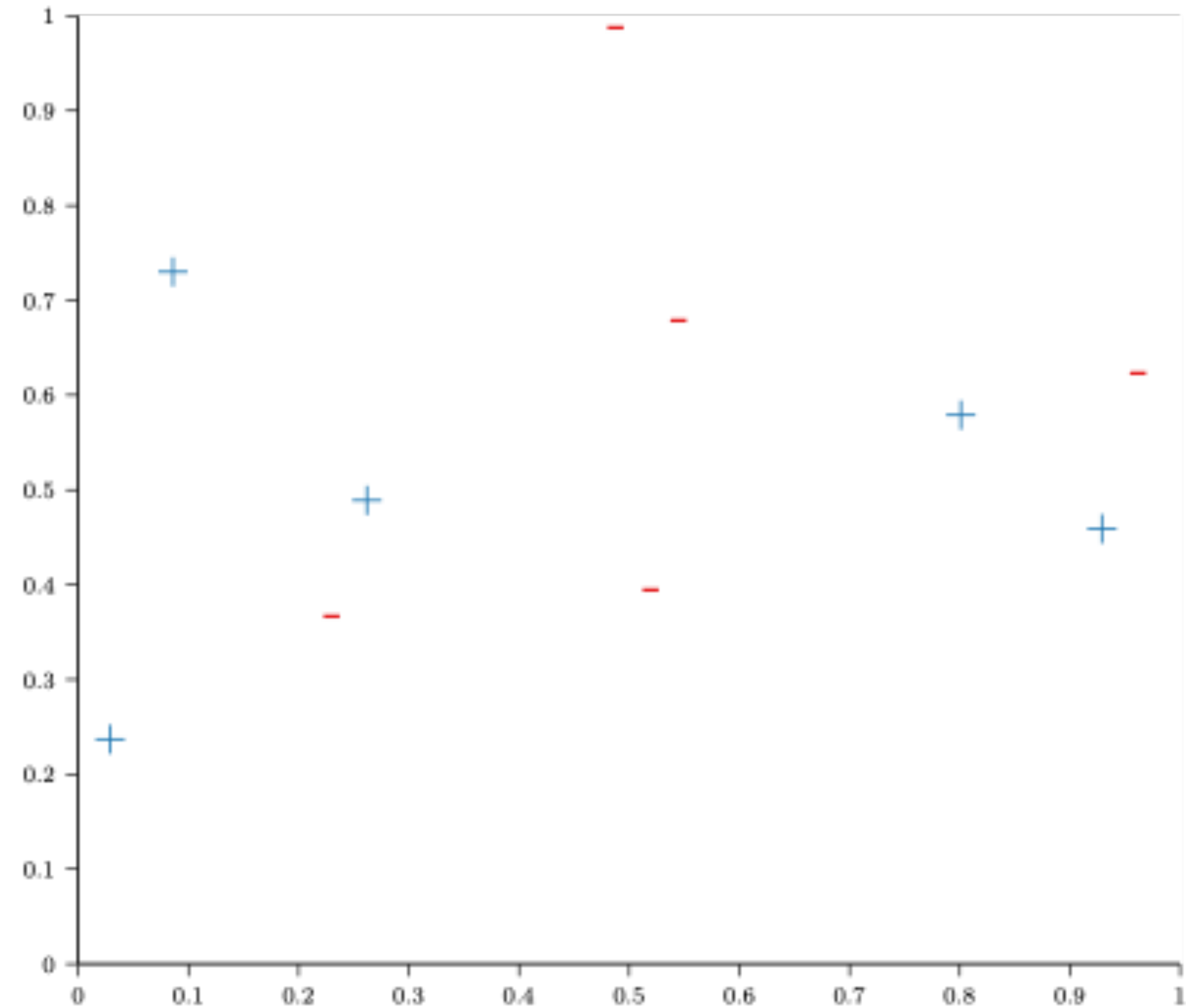
Neighbors

- Predictions for nearby points tend to be similar
- Motivates ***nearest-neighbor*** methods: predict based on nearby training points
 - ▶ often not SOTA accuracy
 - ▶ but highly interpretable
- NN methods form the basis for lots of AI systems
 - ▶ e.g., search or recommendation systems
 - ▶ e.g., related to attention in transformers: store some keys, retrieve best match(es) for a given query
 - ▶ useful if we don't just need to predict a label, but return an item (a webpage, a product to buy)

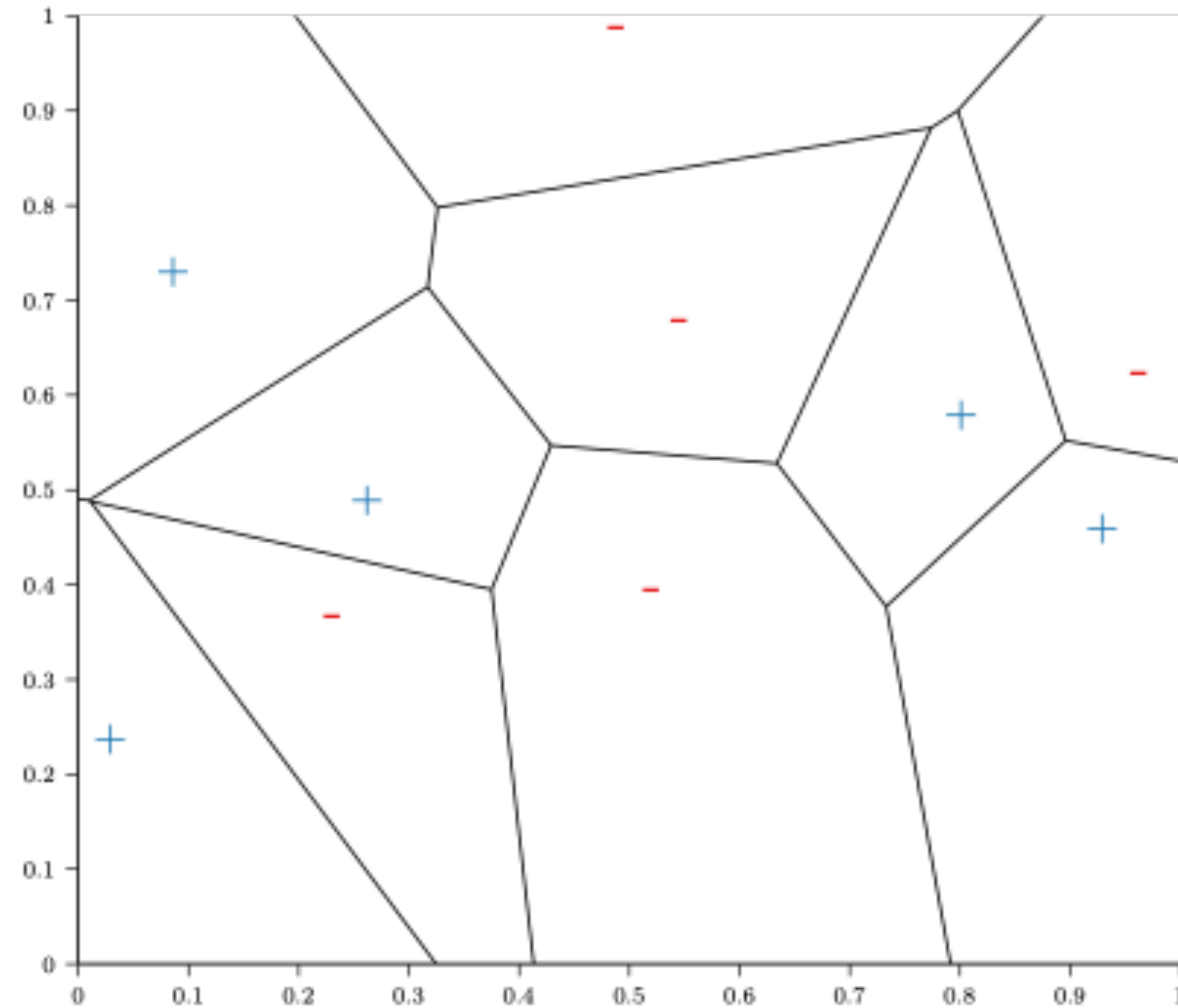
k-nearest-neighbors

- Train: just store the dataset!
 - ▶ no parameters to fit: k -NN is a ***nonparametric*** method
- Test: look up the k nearest training points
 - ▶ for classification, predict the most common label among neighbors (like *majority* classifier but only on neighbors)
 - ▶ for regression, predict the average label
- 1-NN is just called ***nearest neighbor***

Nearest Neighbor: Example

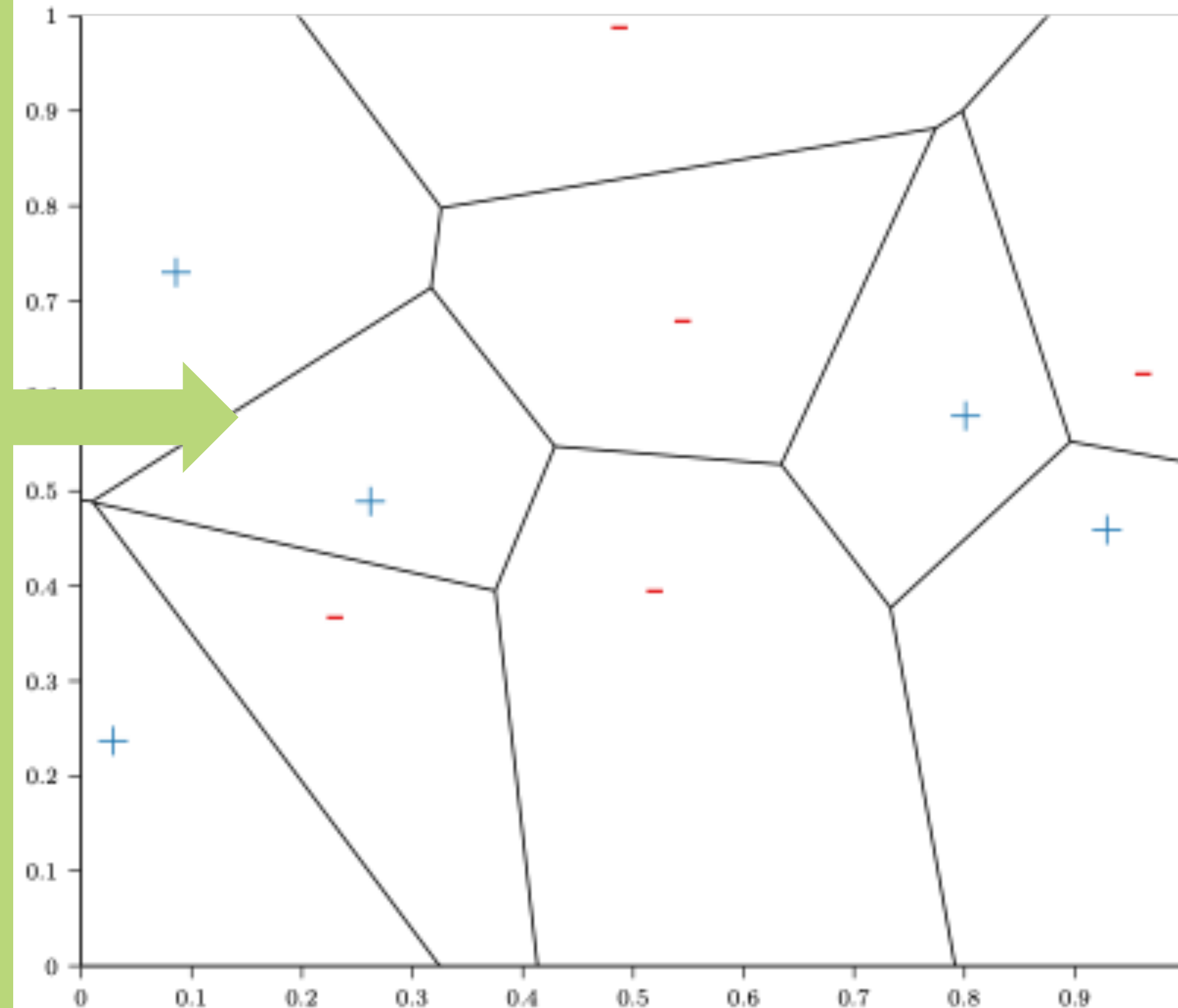


Nearest Neighbor: Example

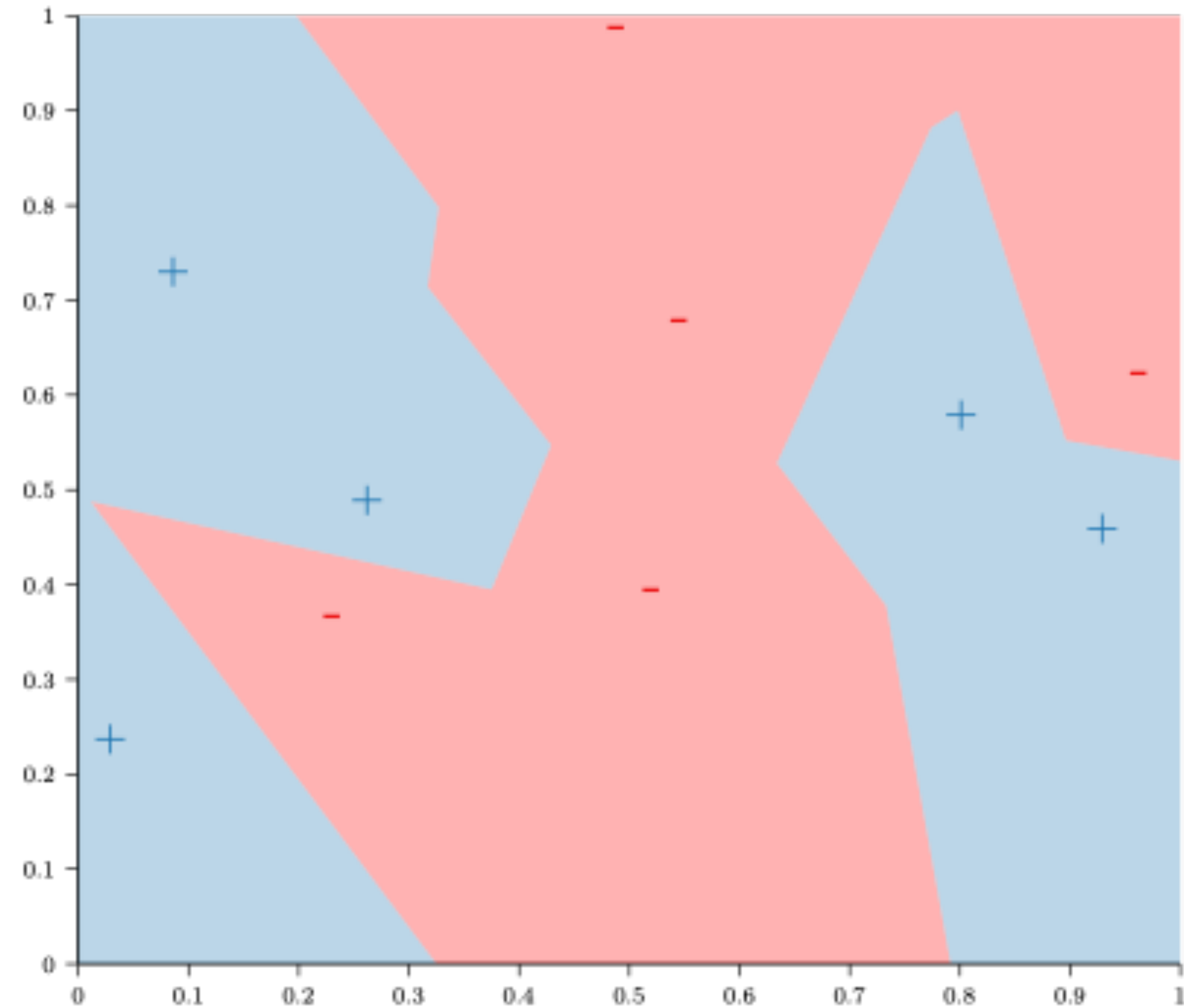


Nearest Neighbor: Example

- This is a **Voronoi diagram**
- Each **cell** contains one of our training examples
- **All points within a cell** are **closer** to that example than to any other example
- **Points on the Voronoi line segments** are **equidistant** to one or more training examples



Nearest Neighbor: Example



Distance metrics

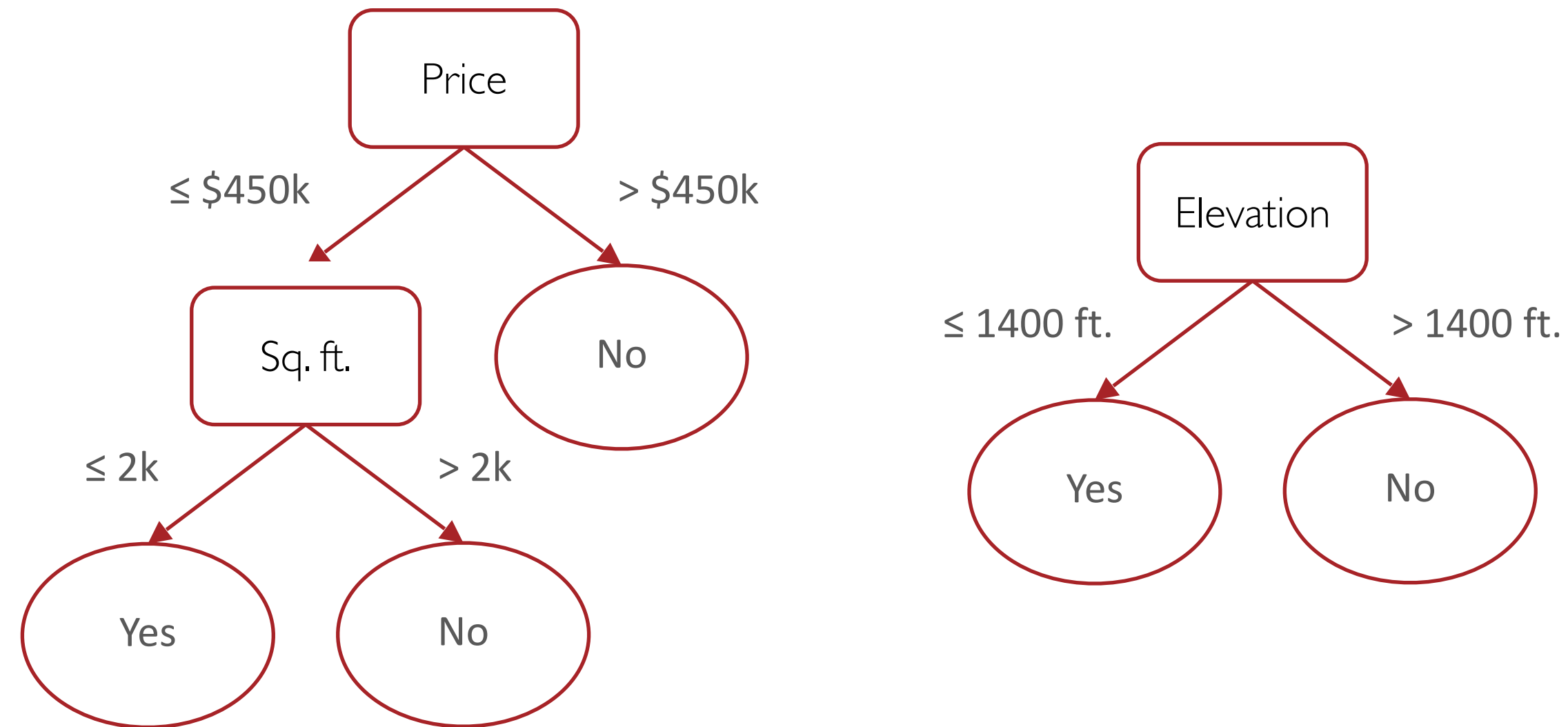
- k -NN has two hyperparameters: number of neighbors k and how we choose to measure distances
- Most common choice is Euclidean or L_2 distance metric:

- ▶ $d(x^{(t)}, x) = \sqrt{\sum_{j=1}^d (x_j^{(t)} - x_j)^2}$

- Many other possibilities — next most common is probably Manhattan or L_1 :

- ▶ $d(x^{(t)}, x) = \sum_{j=1}^d |x_j^{(t)} - x_j|$

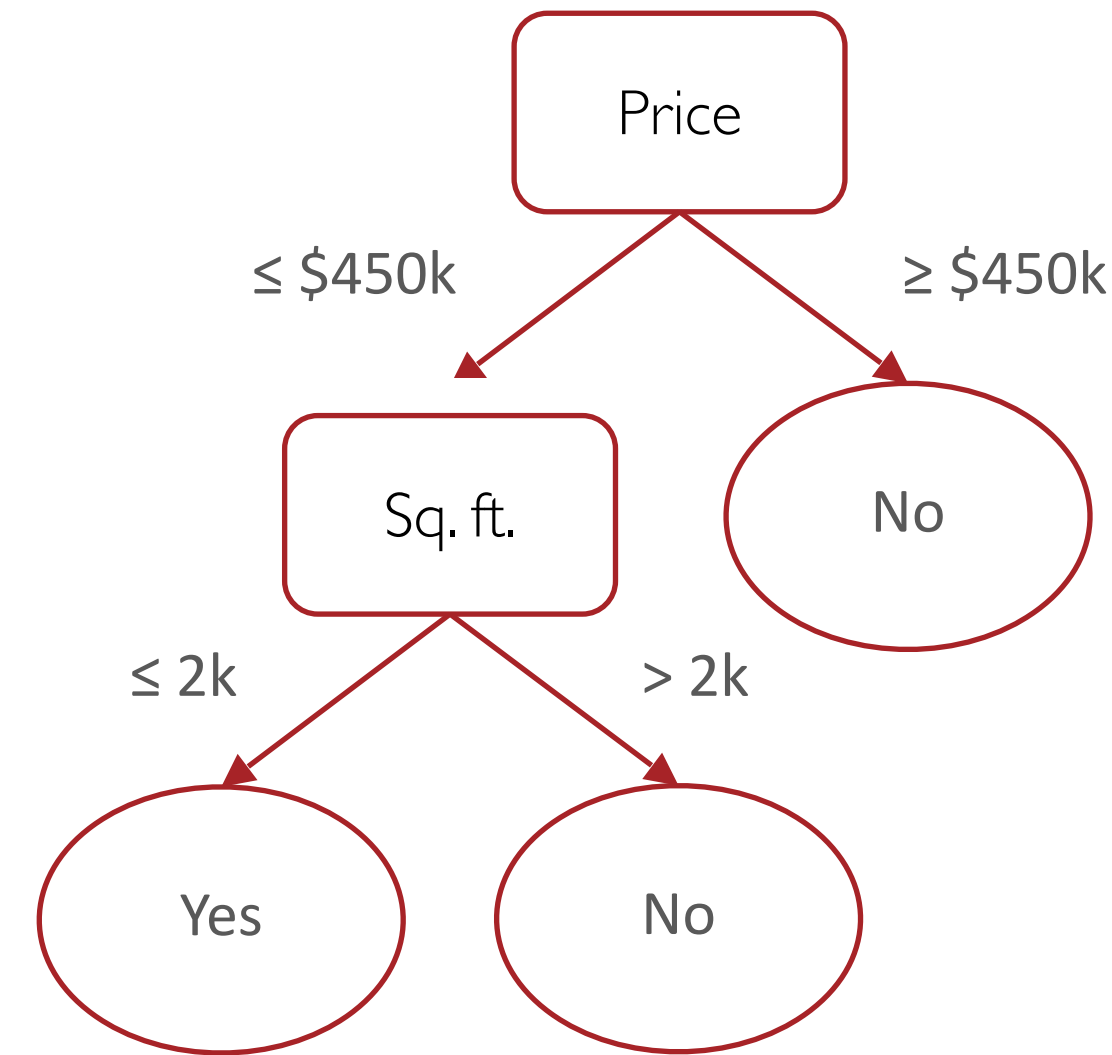
Model selection



- A ***hypothesis space*** $\mathcal{H}$ is a set of hypotheses (classifiers, regression functions, ...)
 - ▶ e.g., all decision trees of depth ≤ 3 with splits on attributes { square feet, price, elevation, rooms }
- The learning algorithm searches through $\mathcal{H}$ to try to find a good hypothesis

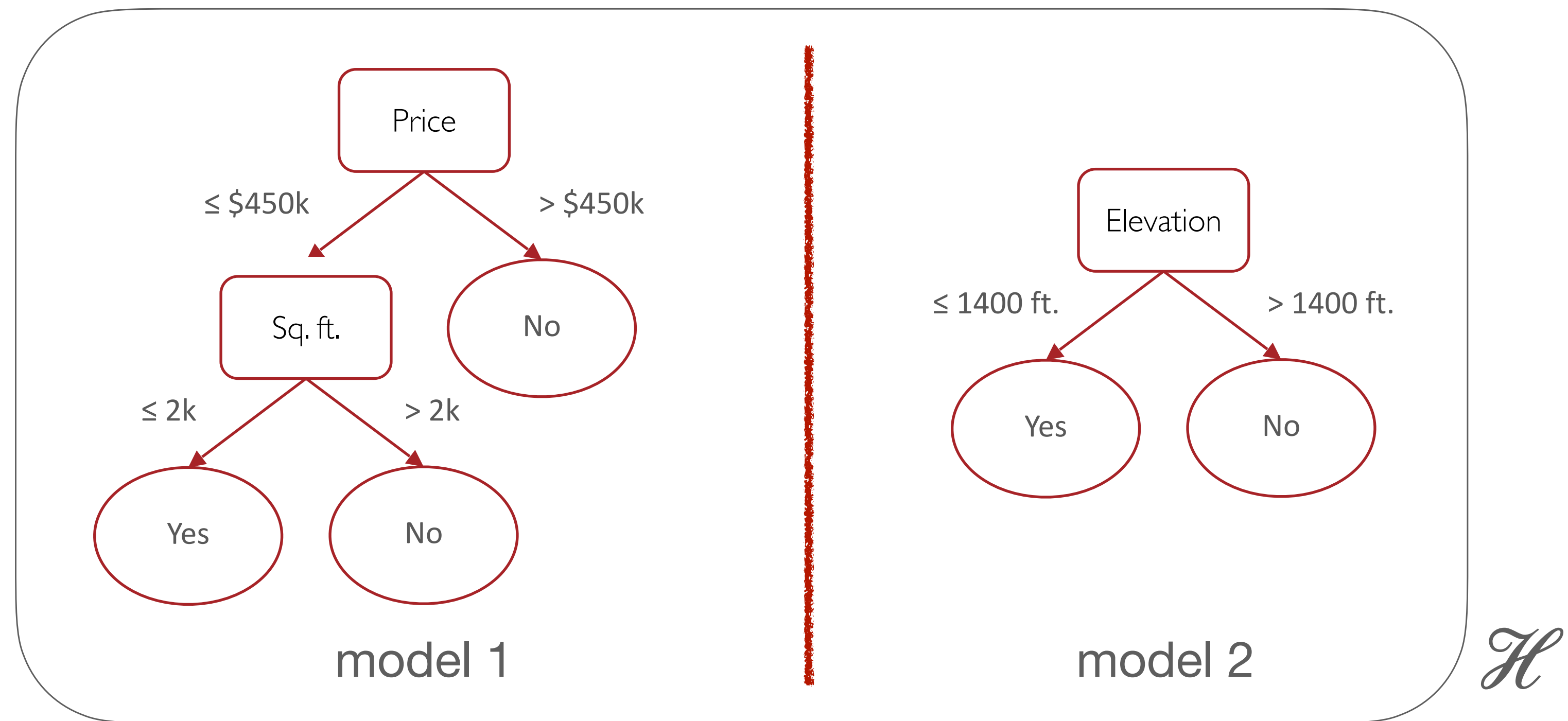
Model selection

recall: **model** can refer to either the class or the specific trained model



- If the hypotheses all share a similar form (e.g., decision trees of same structure), hypothesis space is often called a **model**
 - ▶ different hypotheses within this form (i.e., trained models) are distinguished by **parameters**; it is learner's job to fit these

Model selection



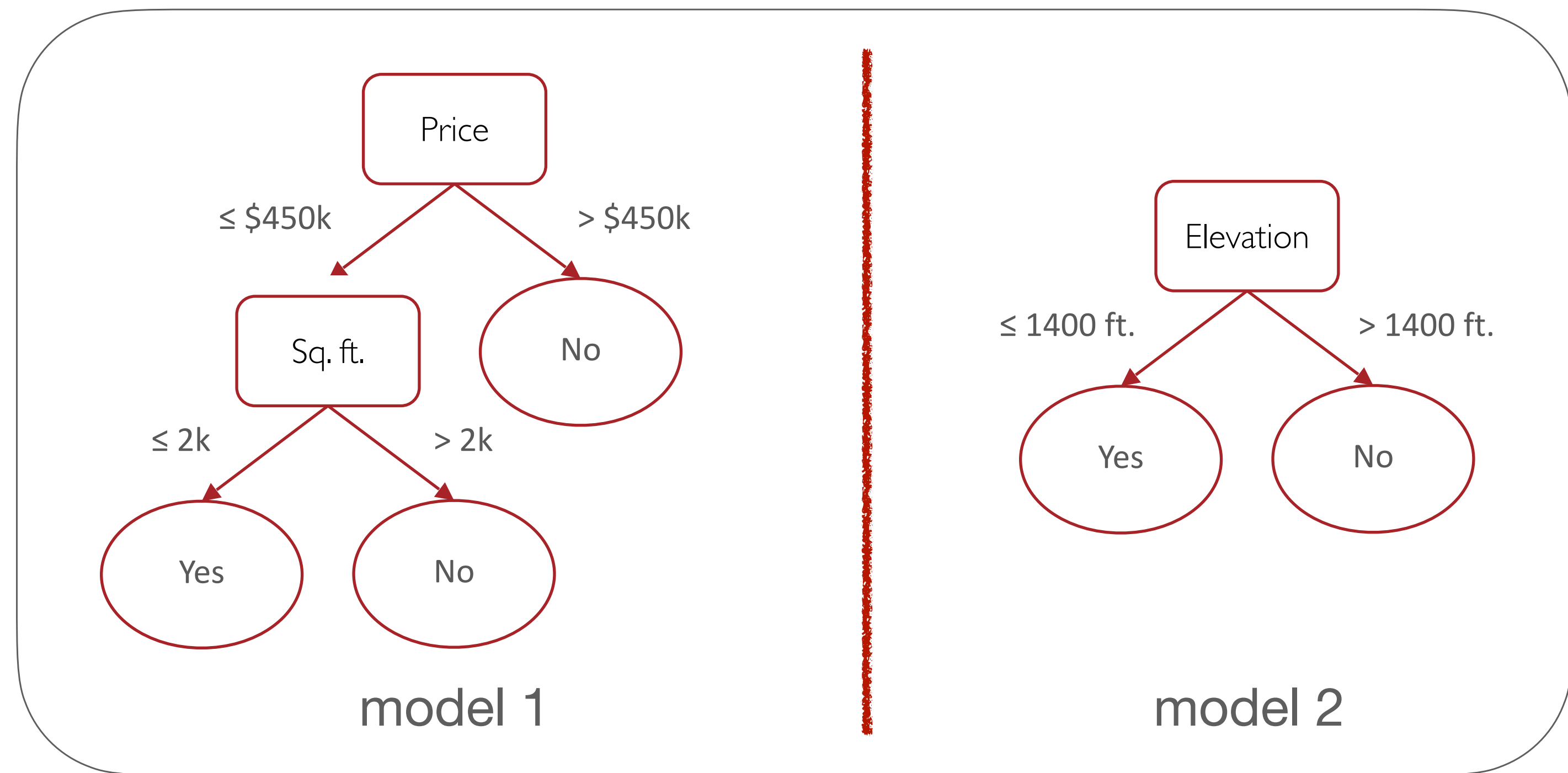
- More generally, may want to search over several models
 - ▶ hypothesis space is the *union* of the models
 - ▶ and *model selection* is the process of choosing
- Why?
 - ▶ trade off simplicity vs. expressiveness — reduce overfitting by taking simplest model that fits

Hyper-parameters

Distinction between parameters vs. hyperparameters is *our* choice: what's important enough to call a new model? What naturally fits in our training/optimization method?

- Often, we call the distinguishing factors among models ***hyperparameters***
 - ▶ e.g., depth of decision tree, k in k -NN, or how many attention heads in transformer
 - ▶ in this case, model selection is ***hyperparameter tuning***
- Not just model structure: learning algorithm can have hyperparameters
 - ▶ e.g., mutual information vs. Gini split criterion for decision tree learner
 - ▶ e.g., momentum parameter for deep net learning
 - ▶ e.g., early stopping for either decision trees or deep nets

Model selection loop



- Often, two nested loops
 - ▶ inner (learning) loop fits parameters of one model
 - ▶ outer (model selection) loop chooses which model

Train/ validation/ test split for model selection

- Split data $\mathcal{D}$ into three parts:
 - ▶ $\mathcal{D}_{\text{train}}$: learning *within a model class*
 - ▶ $\mathcal{D}_{\text{val}}$: *model selection*
 - ▶ $\mathcal{D}_{\text{test}}$: final evaluation
- Tension: make one bigger to make its result more accurate, at expense of rest

features			labels
Family History	Resting Blood Pressure	Cholesterol	Heart Disease?
Yes	Low	Normal	No
No	Medium	Normal	No
No	Low	Abnormal	Yes
Yes	Medium	Normal	Yes
Yes	High	Abnormal	Yes
No	Low	Normal	No
No	Medium	Normal	No
Yes	Medium	Abnormal	Yes

Model selection loop

- For each model in $\mathcal{H}$:
 - ▶ train on $\mathcal{D}_{\text{train}}$
 - ▶ evaluate on $\mathcal{D}_{\text{val}}$; track best

good parameters, maybe not best, and optimistic performance estimate

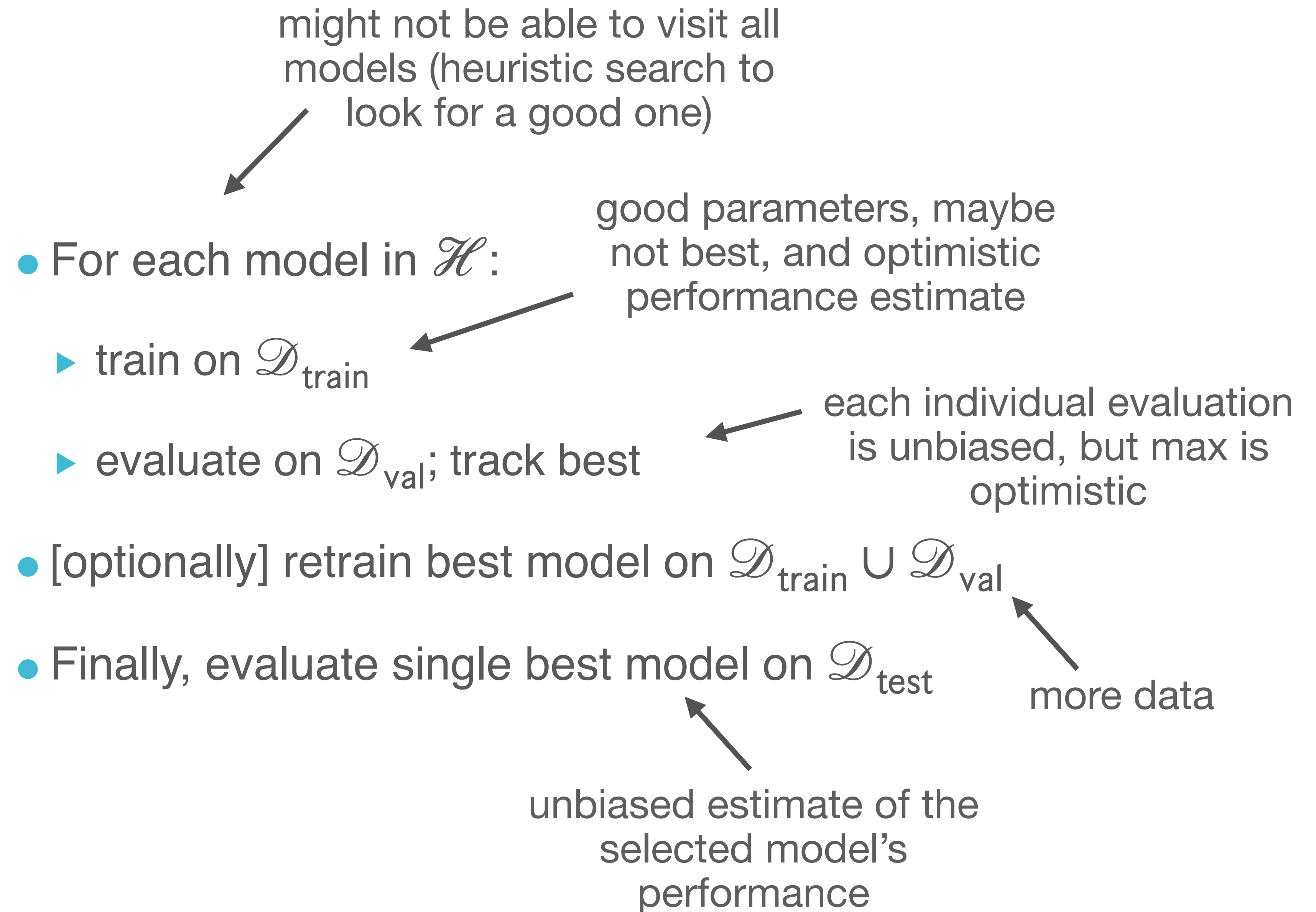
each individual evaluation is unbiased, but max is optimistic
- Finally, evaluate single best model on $\mathcal{D}_{\text{test}}$

unbiased estimate of the selected model's performance

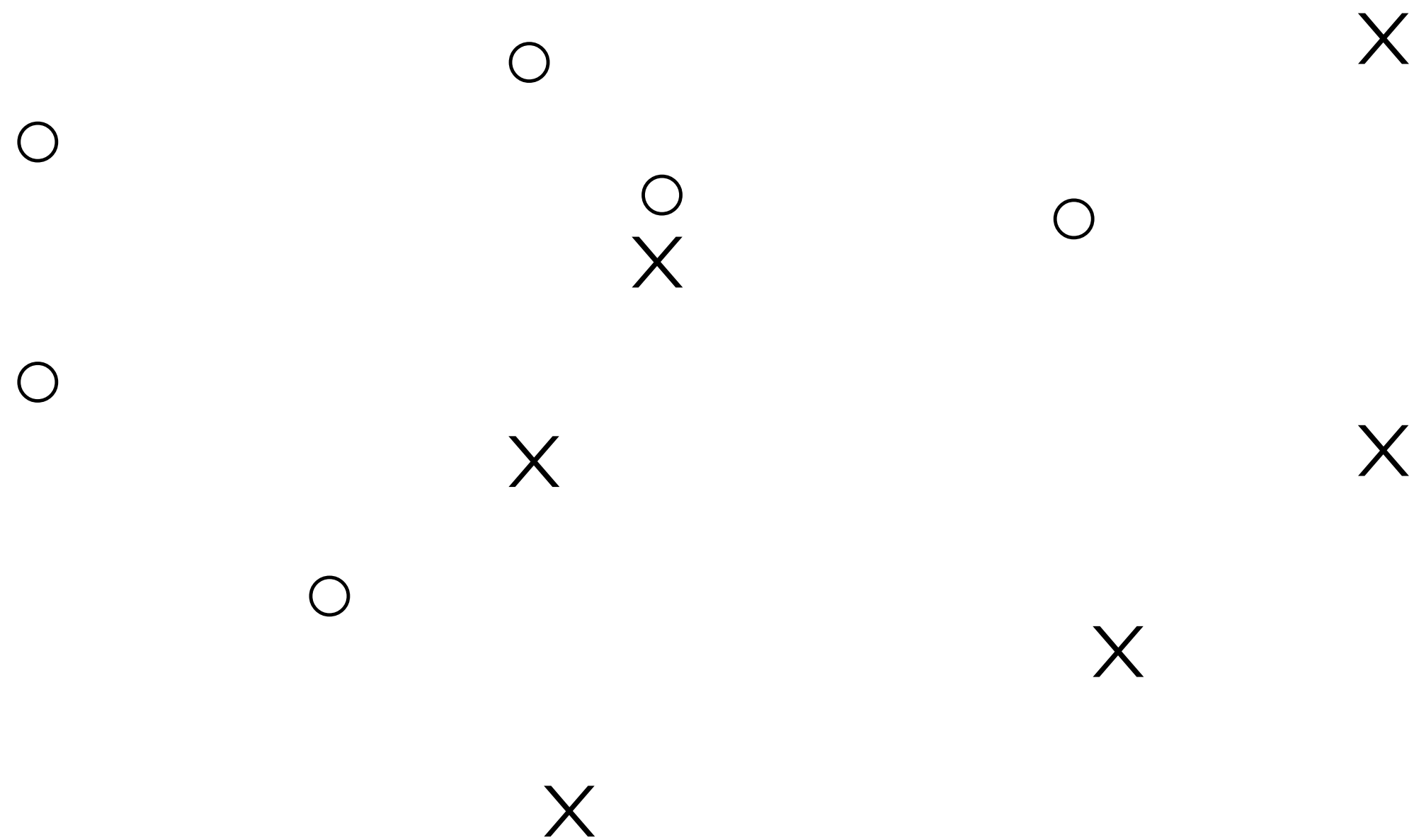
Model selection loop

- For each model in $\mathcal{H}$:
 - ▶ train on $\mathcal{D}_{\text{train}}$
 - ▶ evaluate on $\mathcal{D}_{\text{val}}$; track best
 - [optionally] retrain best model on $\mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{val}}$
 - Finally, evaluate single best model on $\mathcal{D}_{\text{test}}$
- good parameters, maybe not best, and optimistic performance estimate
- each individual evaluation is unbiased, but max is optimistic
- more data
- unbiased estimate of the selected model's performance

Model selection loop

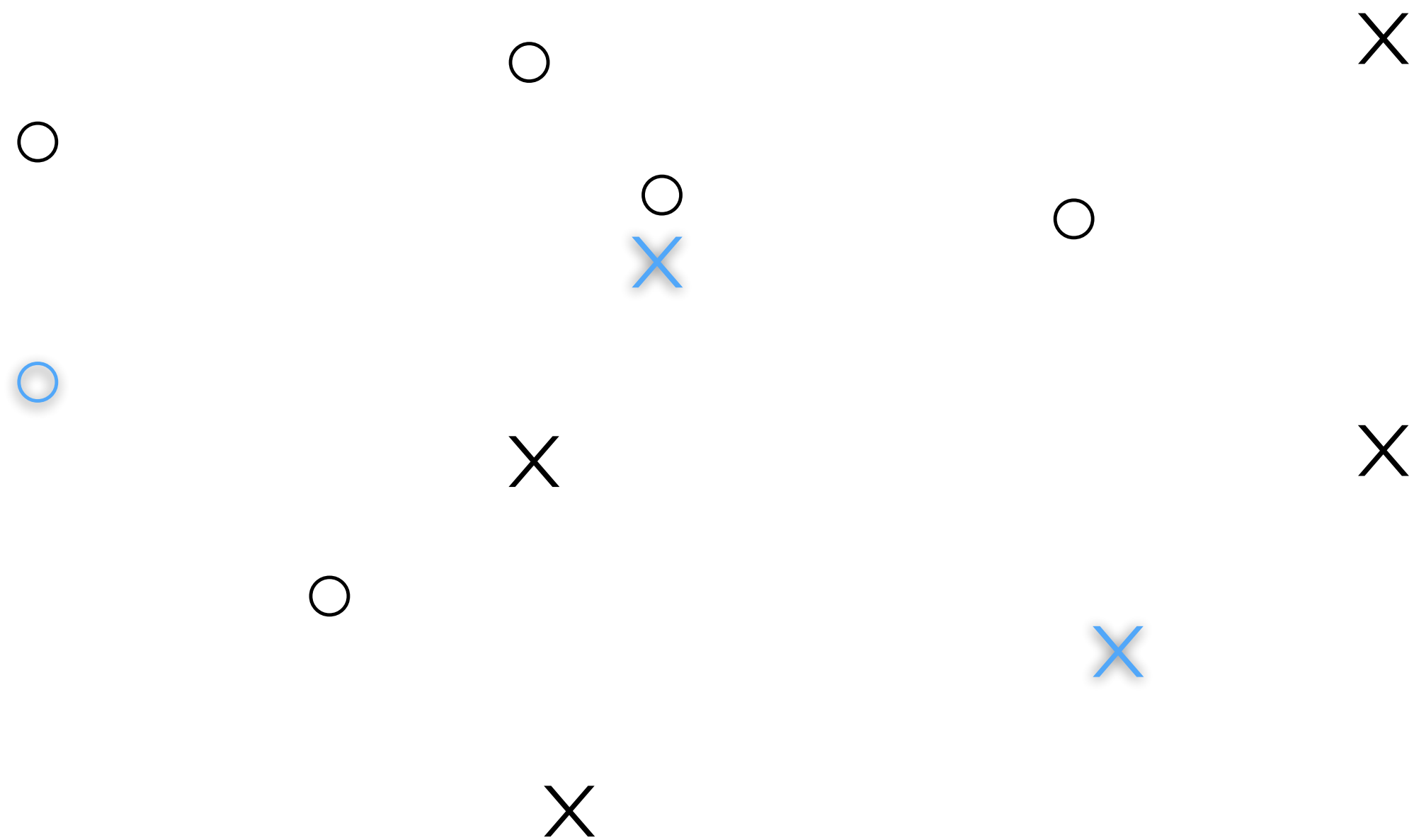


***Inner loop:
train and
evaluate a
single model***



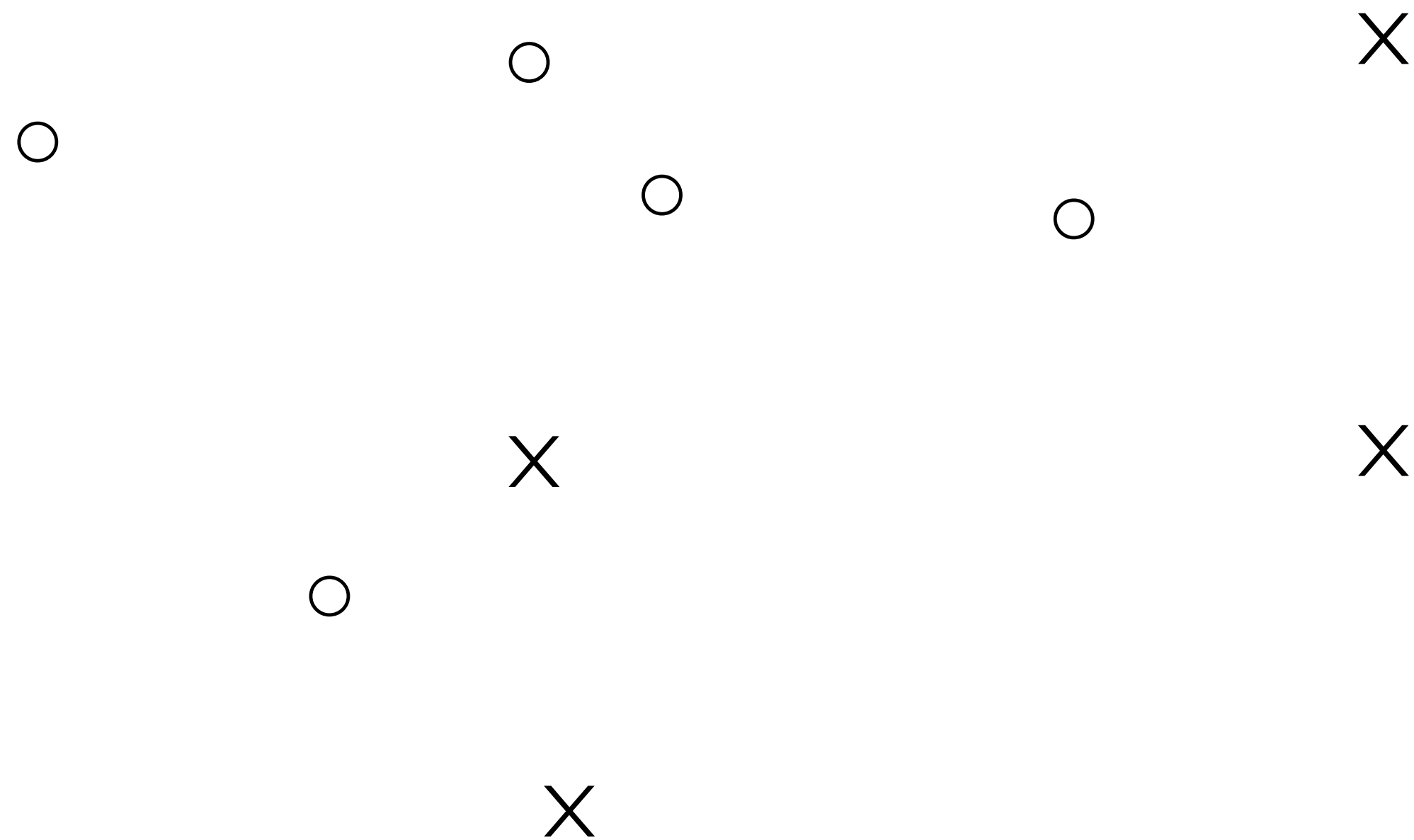
training set

***Inner loop:
train and
evaluate a
single model***



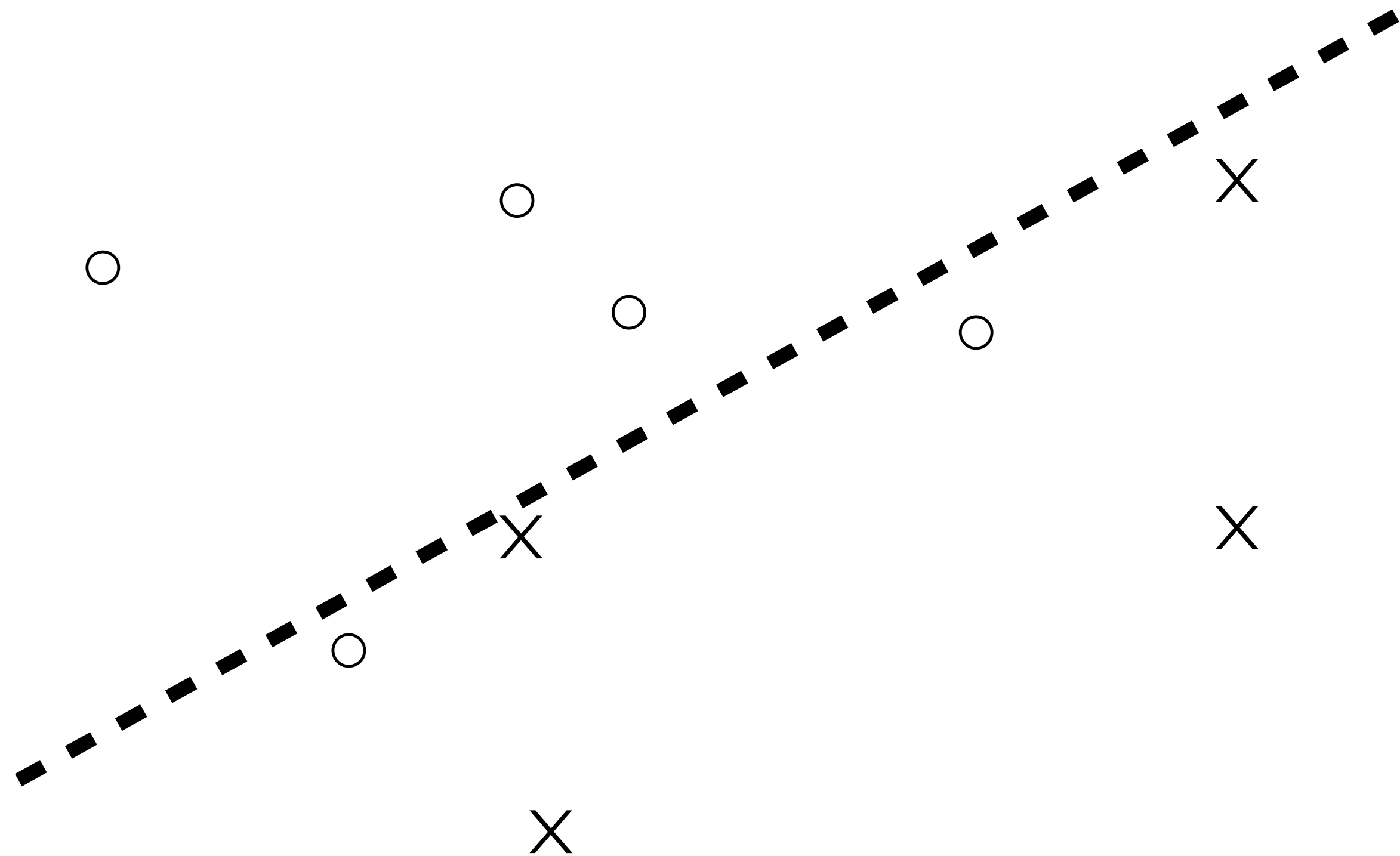
remove hold-out group, fit on rest

***Inner loop:
train and
evaluate a
single model***



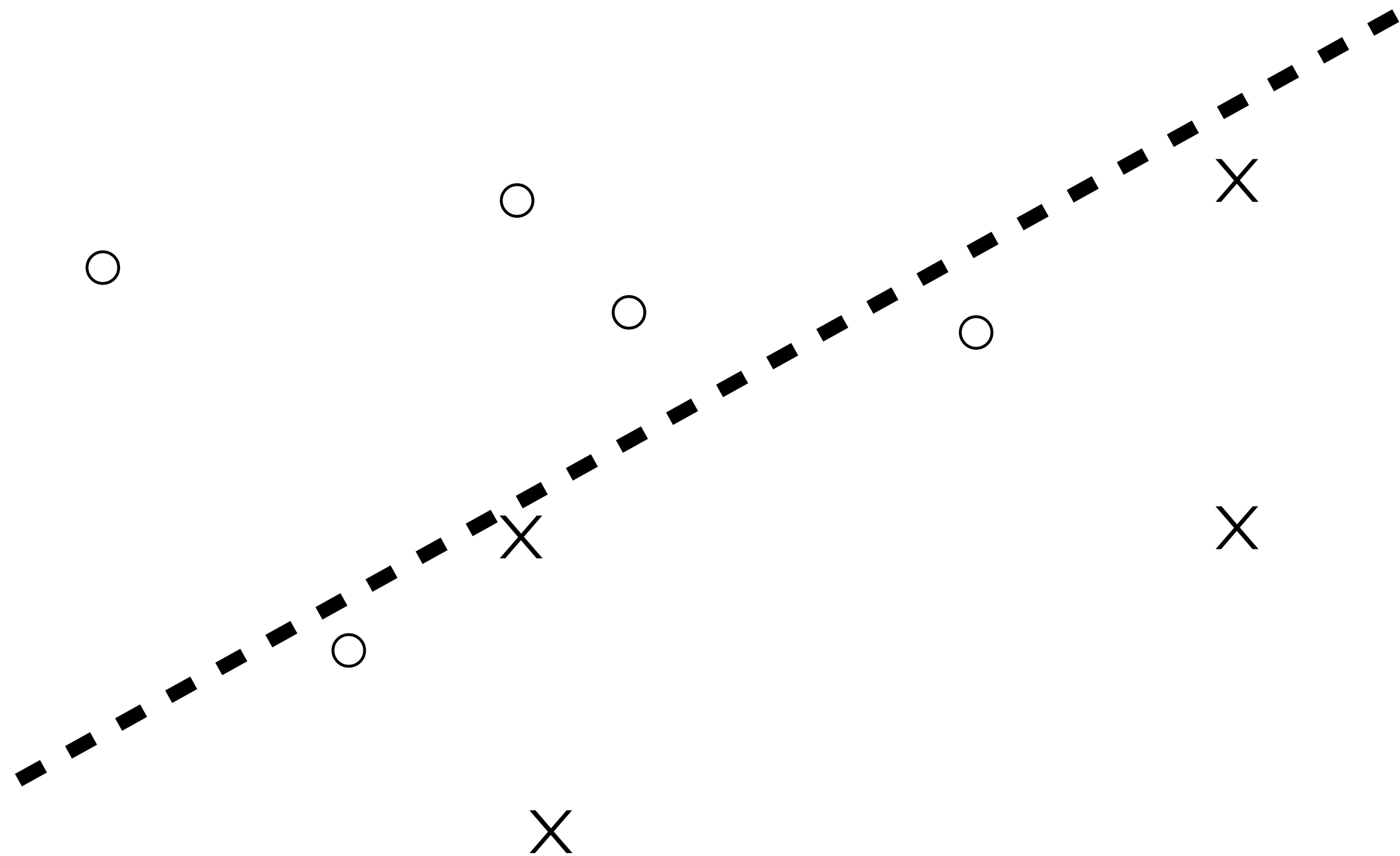
remove hold-out group, fit on rest

***Inner loop:
train and
evaluate a
single model***



remove hold-out group, fit on rest

***Inner loop:
train and
evaluate a
single model***

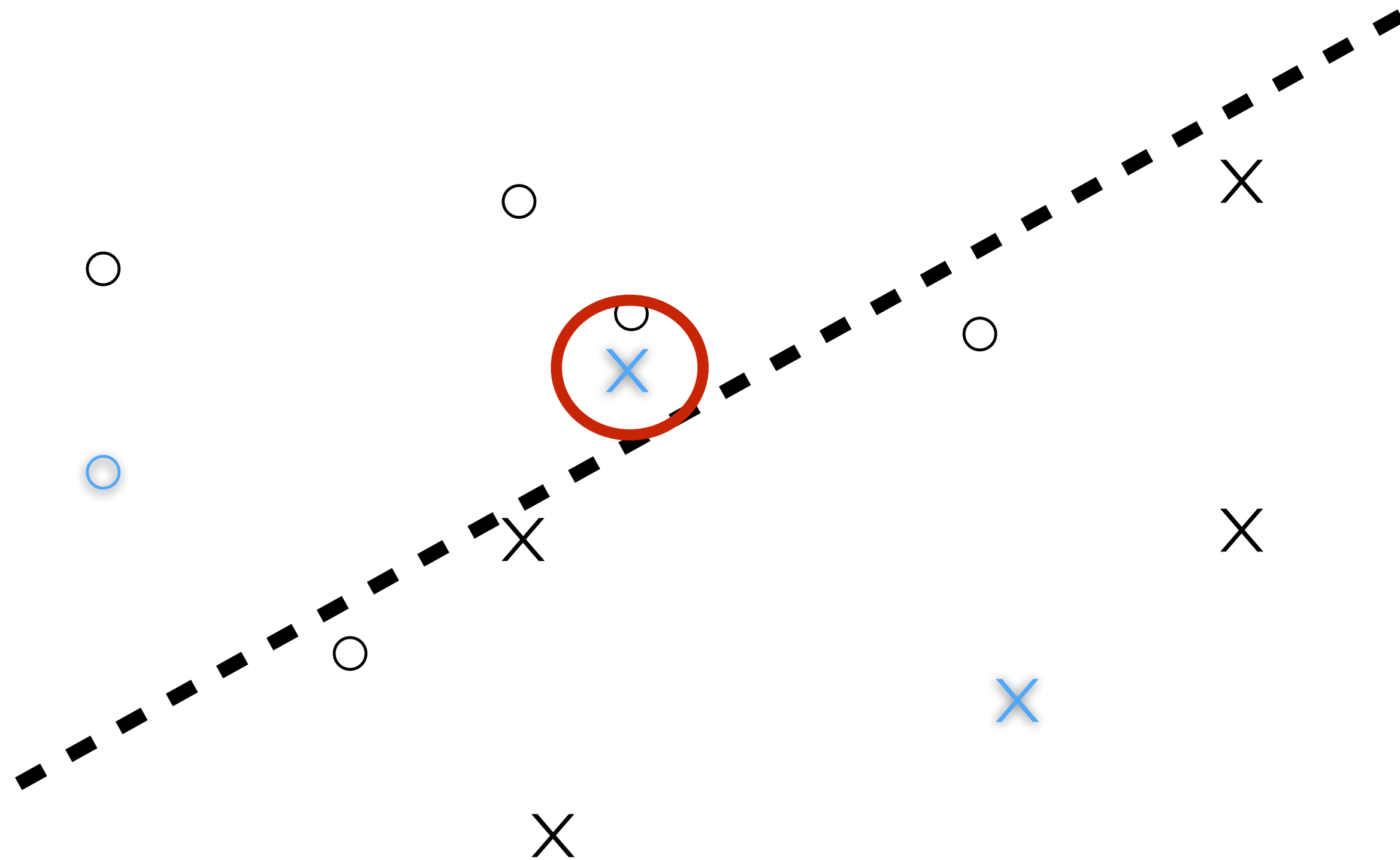


compute error on holdout set

[Faint, illegible handwritten text]

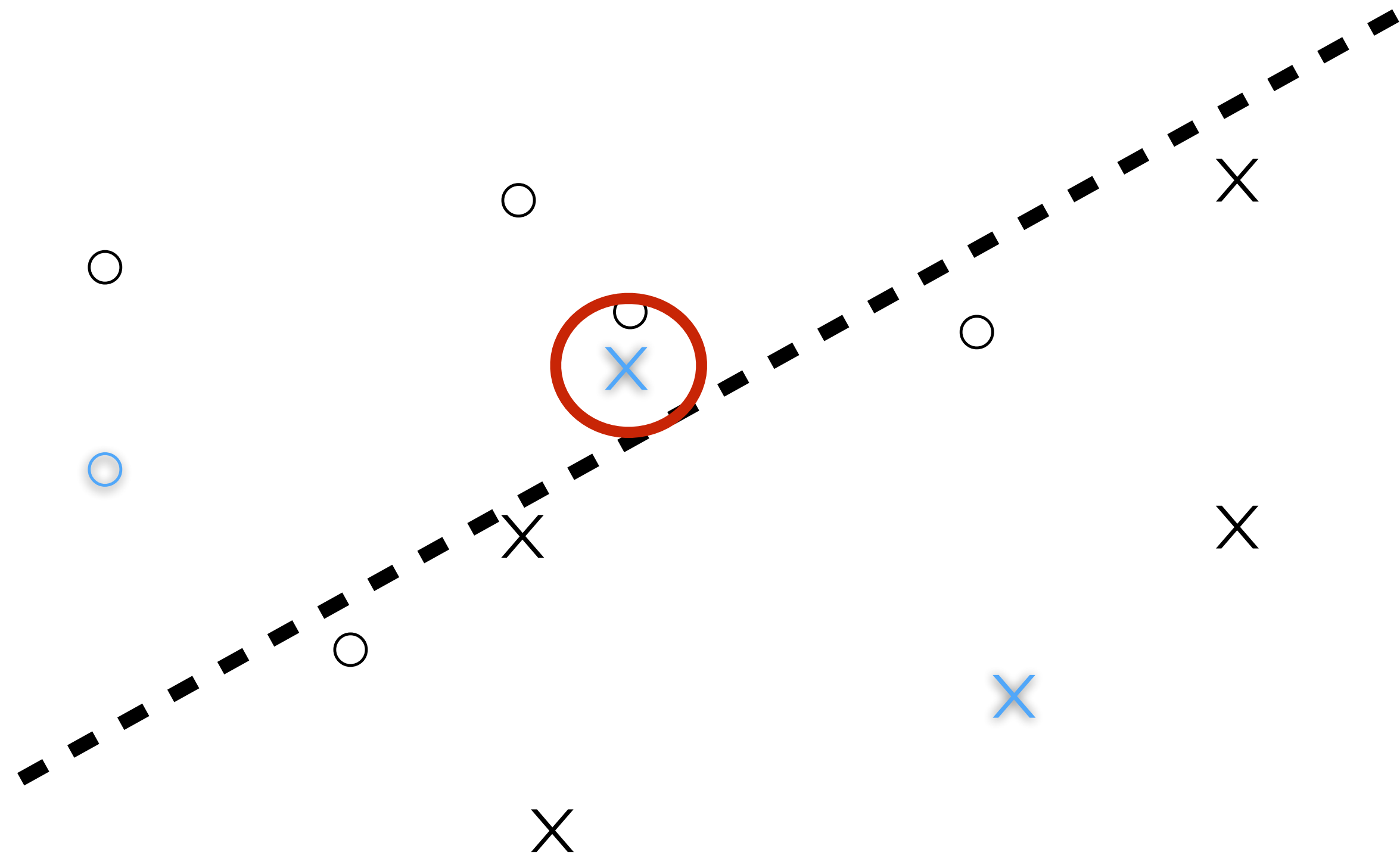


***Inner loop:
train and
evaluate a
single model***



compute error on holdout set

***Inner loop:
train and
evaluate a
single model***



estimated error rate: 1/3

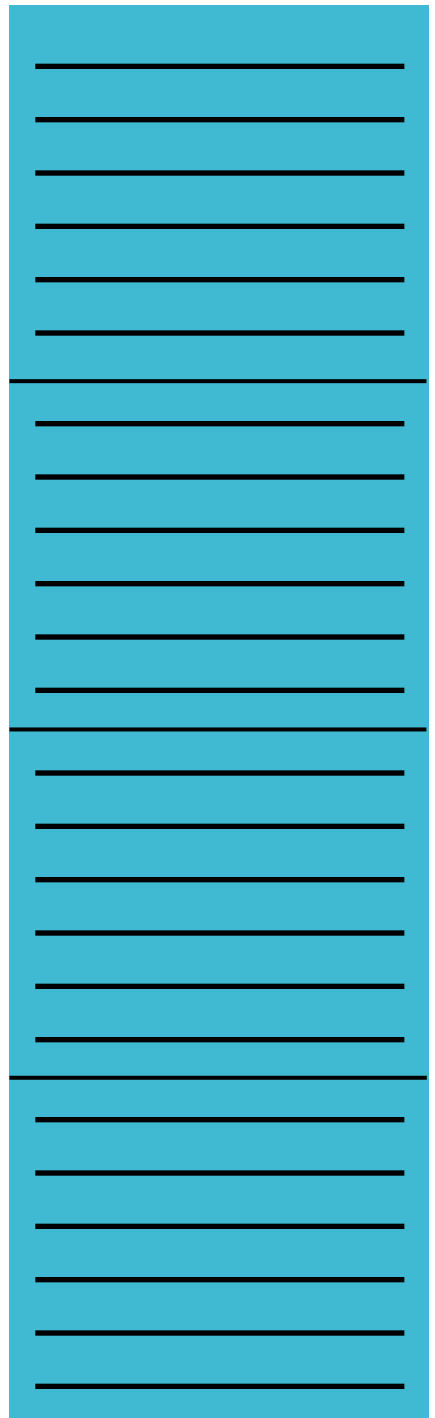
compute error on holdout set

Problem with hold- out set

- Tension between size of $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{val}}$
 - ▶ $\mathcal{D}_{\text{train}}$ is slightly too small
 - ▶ often $\mathcal{D}_{\text{val}}$ winds up small enough to be an inaccurate way to select a model
- What if we could put each training example in both sets?

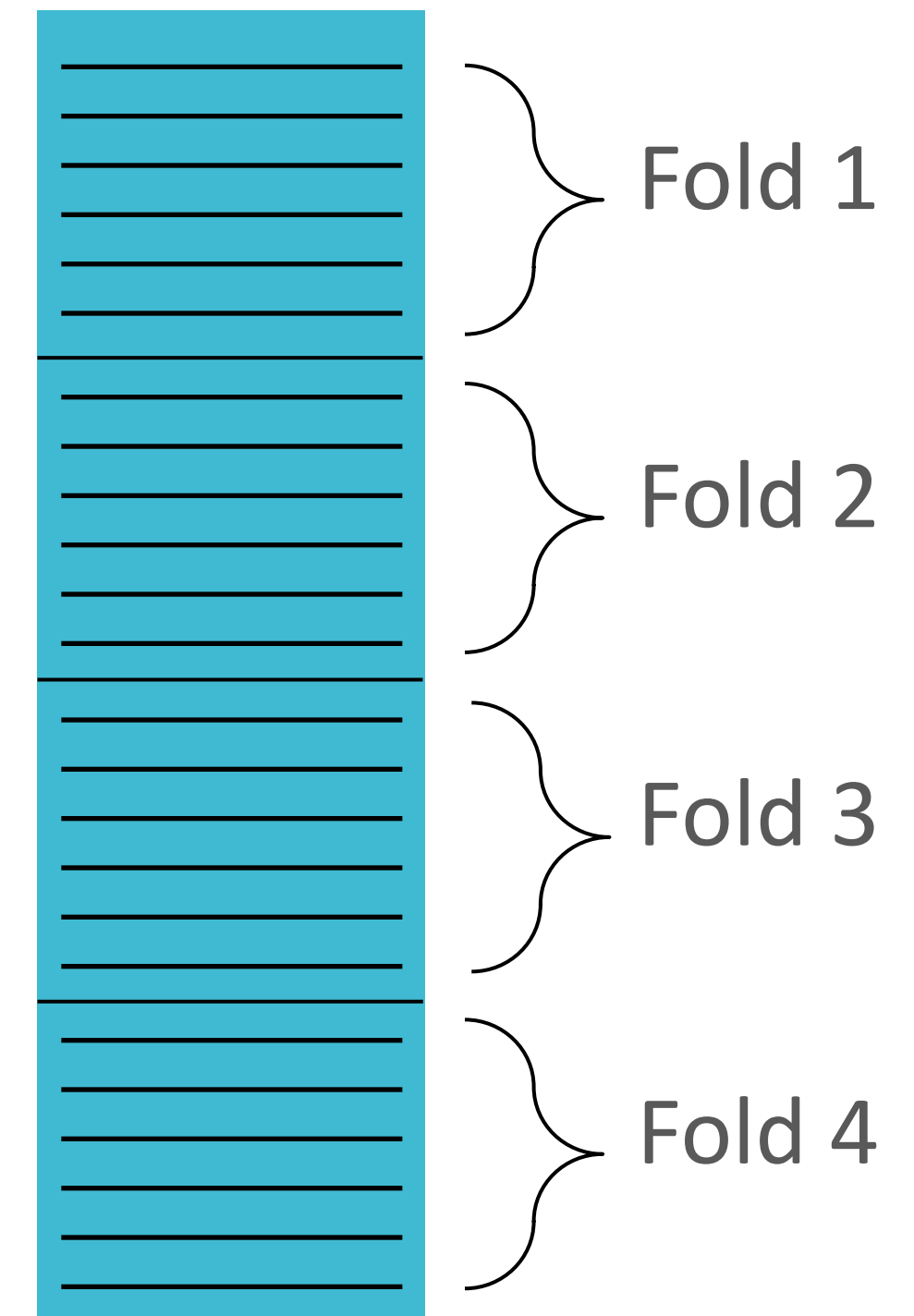
Cross-validation

- Split data into k parts (called ***folds***), repeatedly train on some folds and validate on others
 - ▶ each fold serves as validation for each other fold, hence ***cross***-validation
 - ▶ hyperparameter k trades cost vs. {larger training set, more independent evaluations}



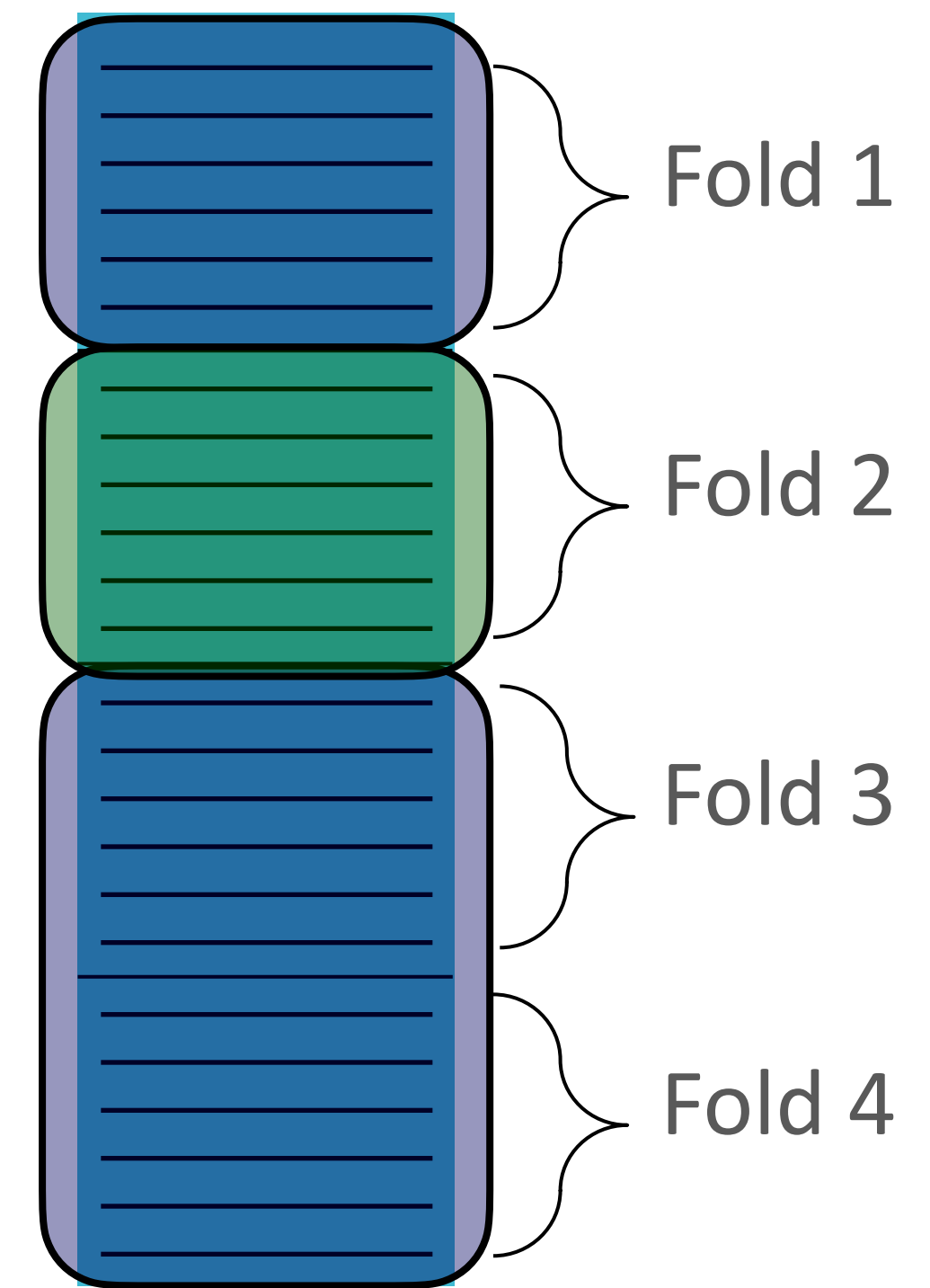
Cross-validation

- Split data into k parts (called ***folds***), repeatedly train on some folds and validate on others
 - ▶ each fold serves as validation for each other fold, hence ***cross***-validation
 - ▶ hyperparameter k trades cost vs. {larger training set, more independent evaluations}

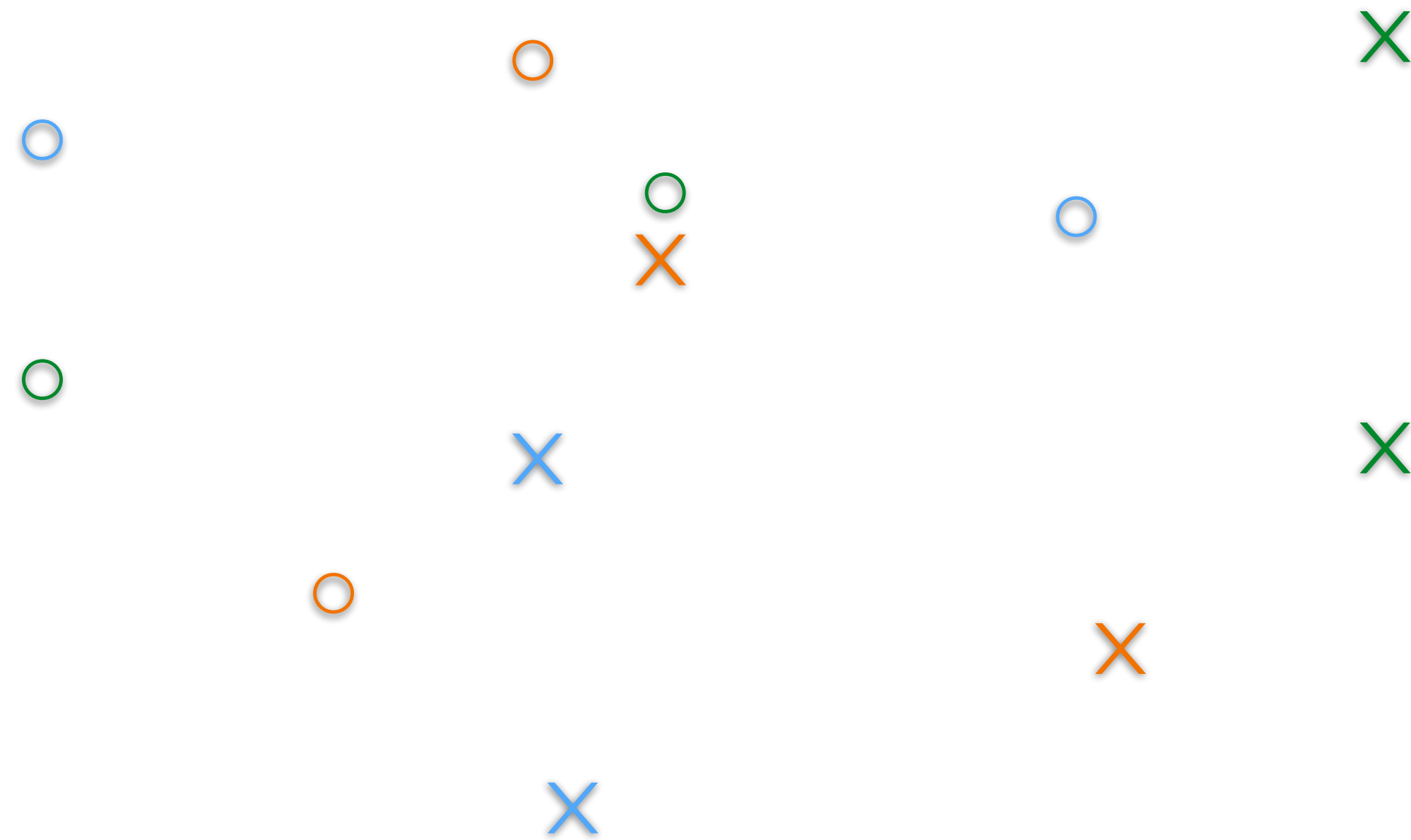


Cross-validation

- Split data into k parts (called ***folds***), repeatedly train on some folds and validate on others
 - ▶ each fold serves as validation for each other fold, hence ***cross***-validation
 - ▶ hyperparameter k trades cost vs. {larger training set, more independent evaluations}

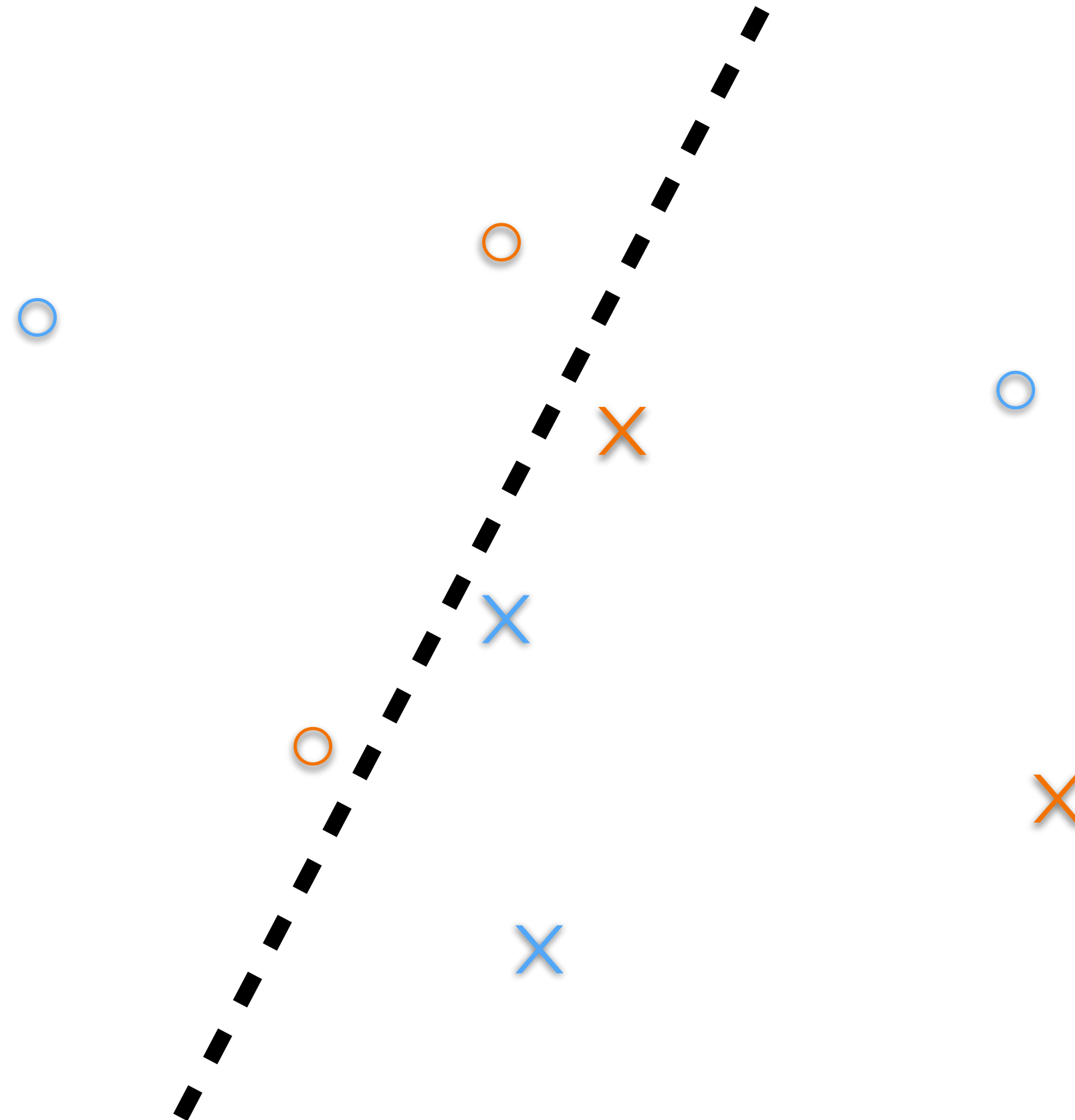


Cross- validation



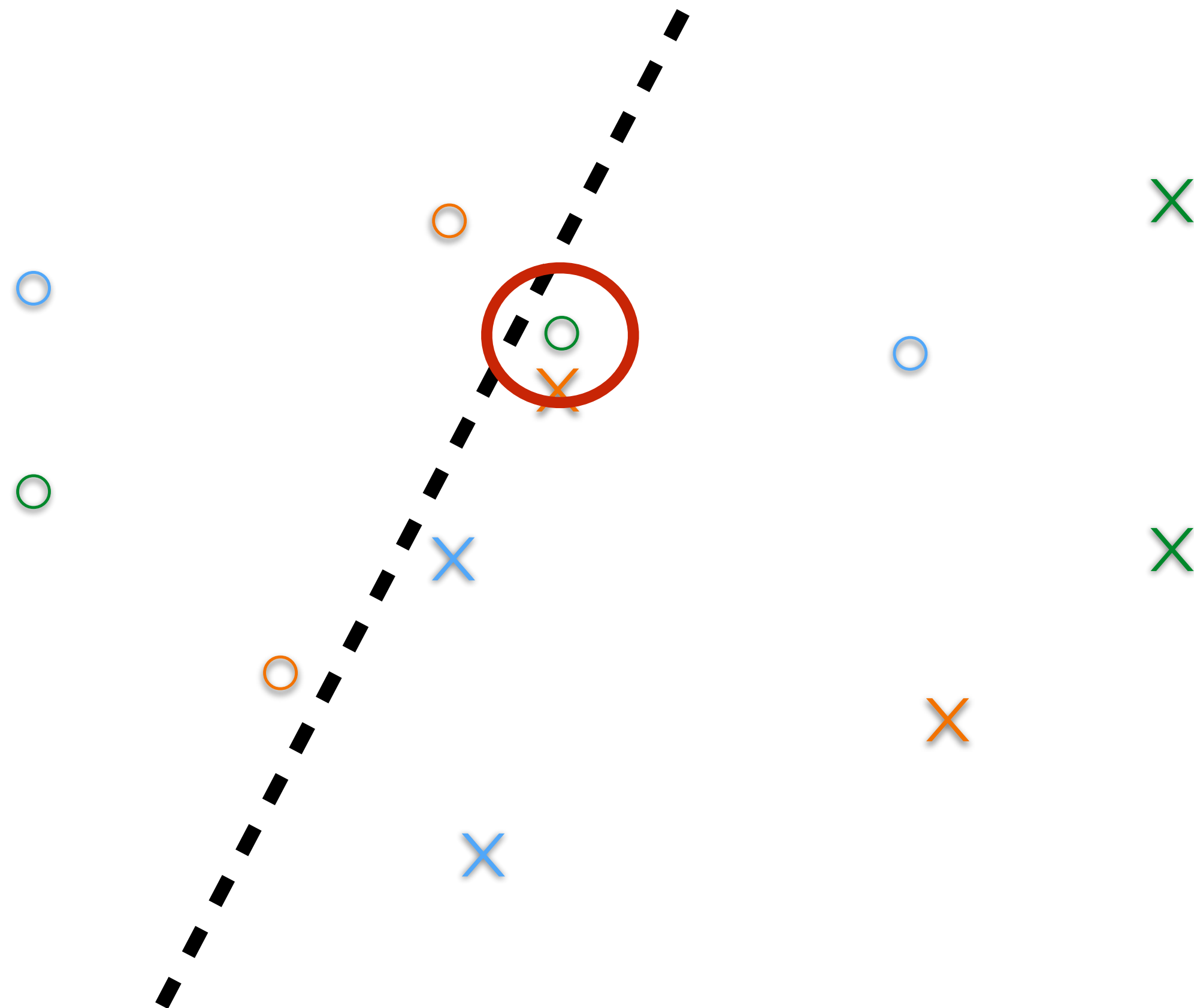
split data evenly into groups (“folds”)

Cross-validation



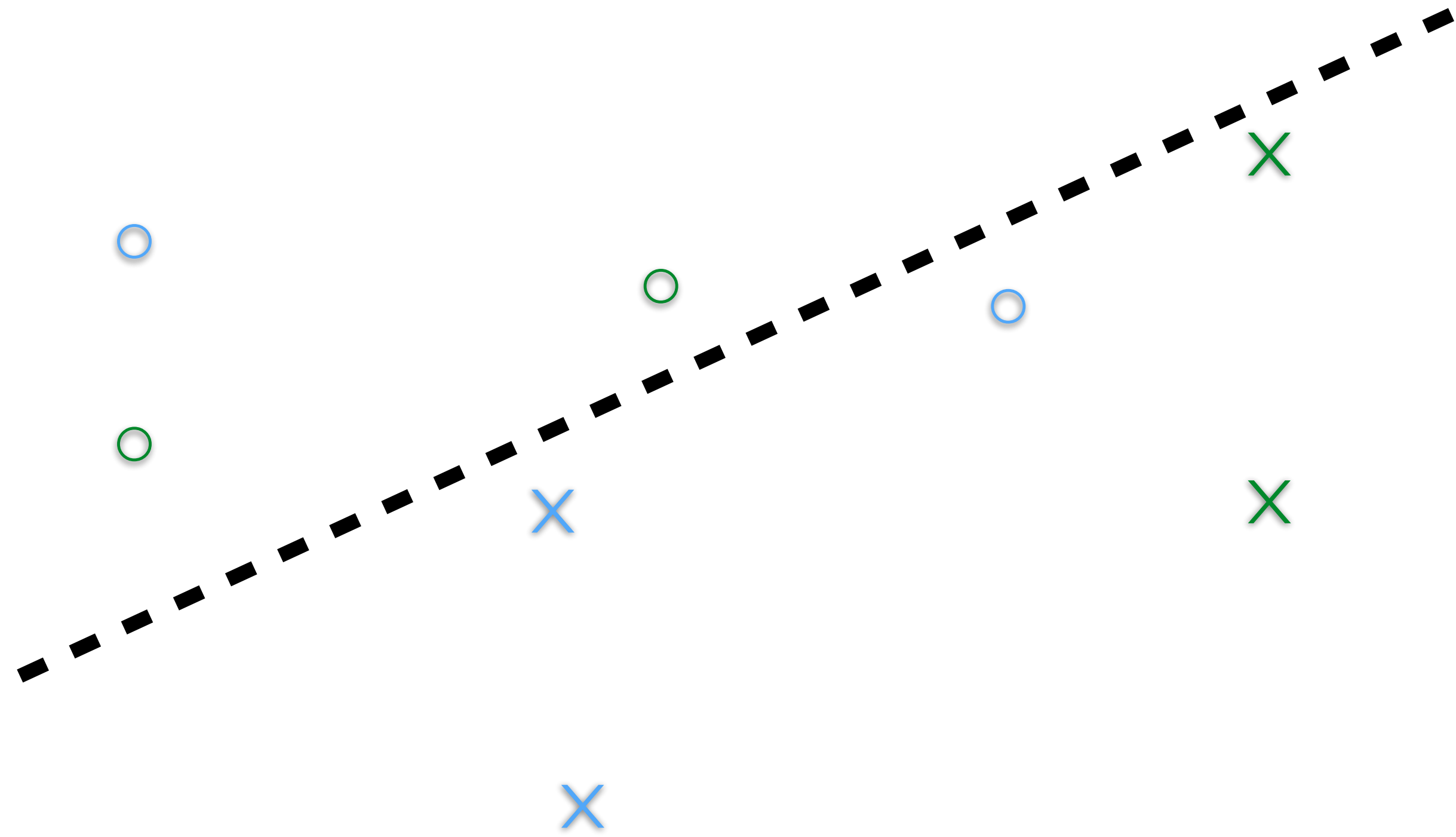
remove green group, fit on rest

Cross-validation



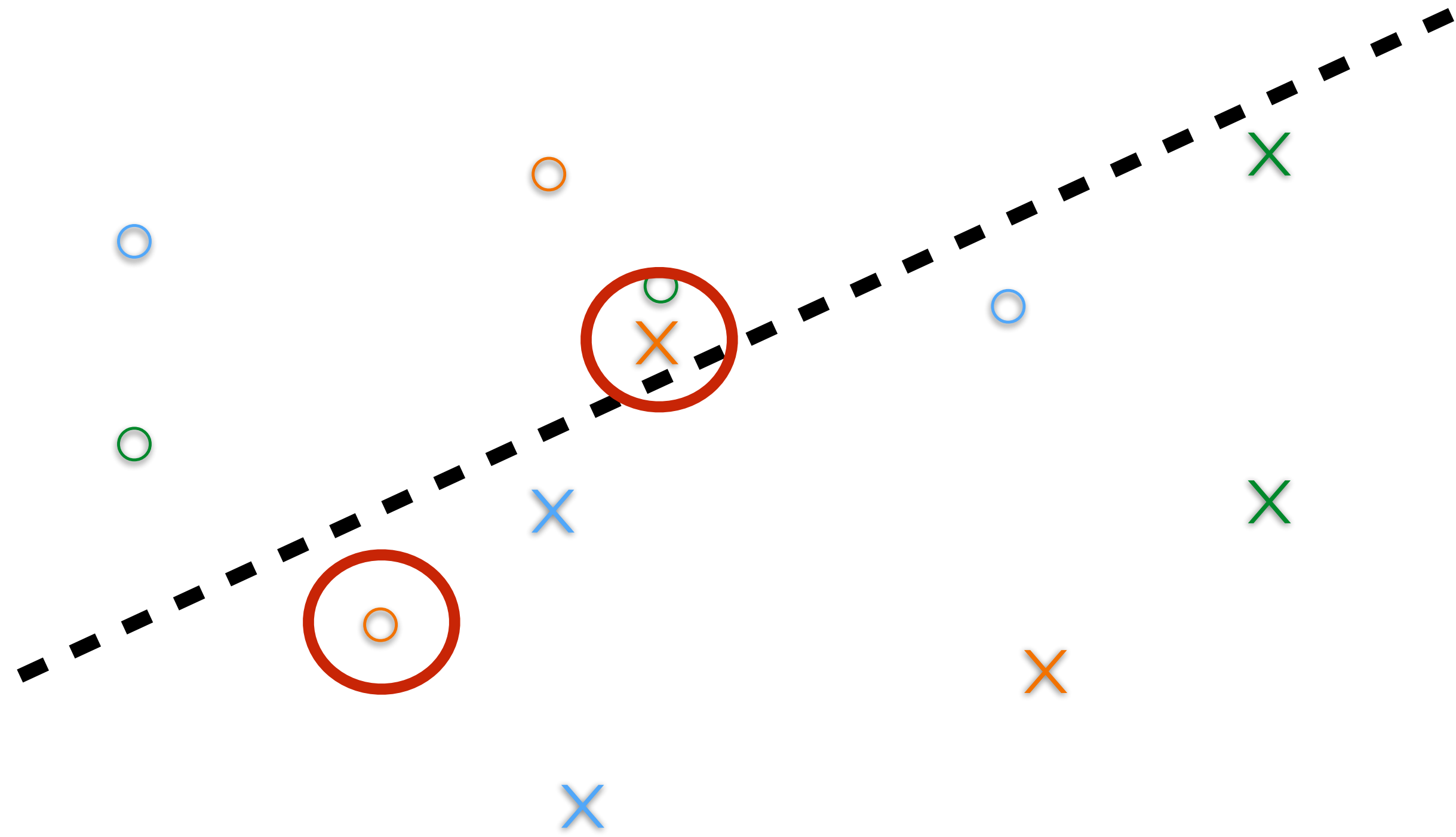
add back green group: error 1/4

Cross-validation



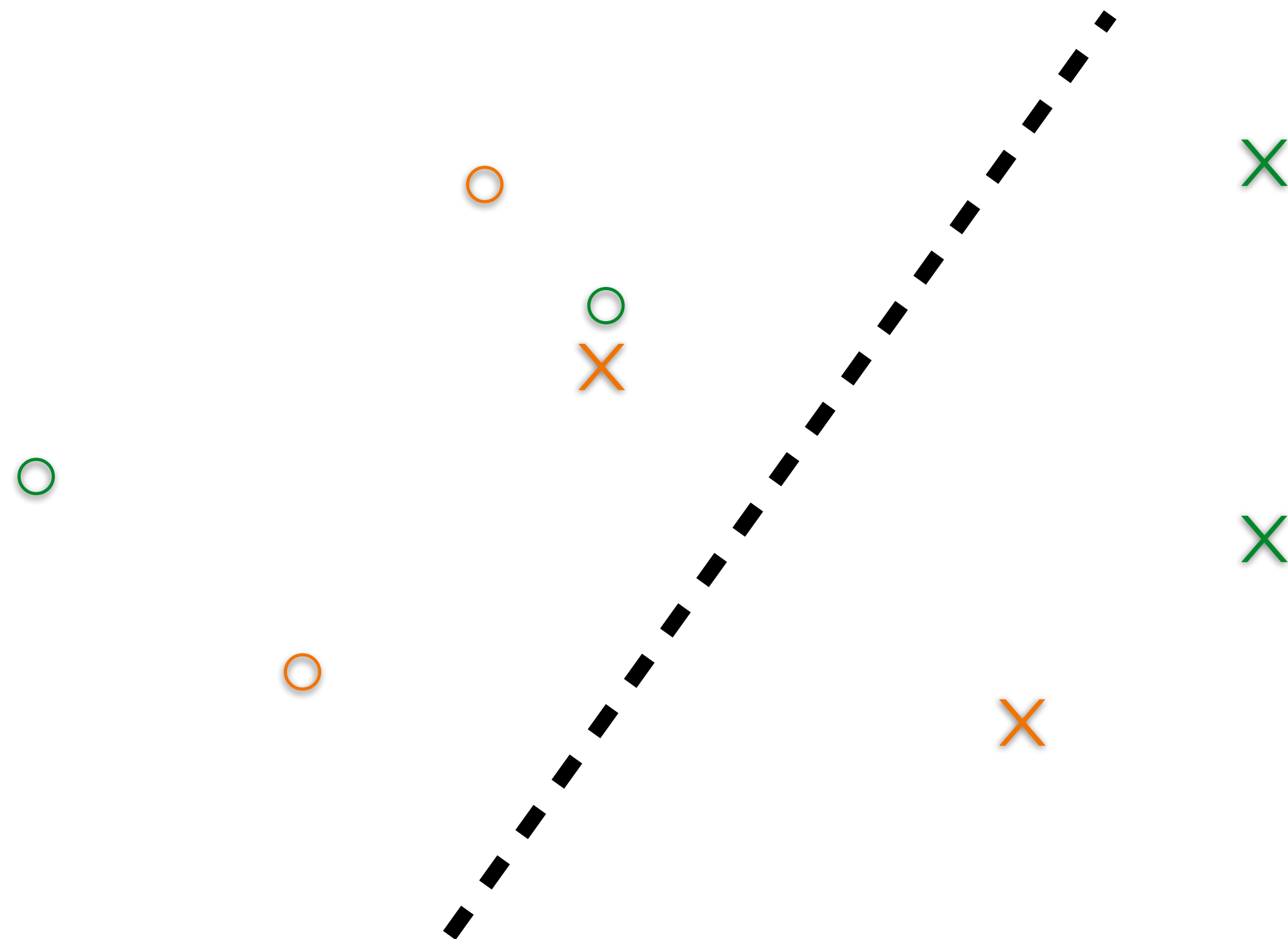
remove orange group, fit on rest

Cross-validation



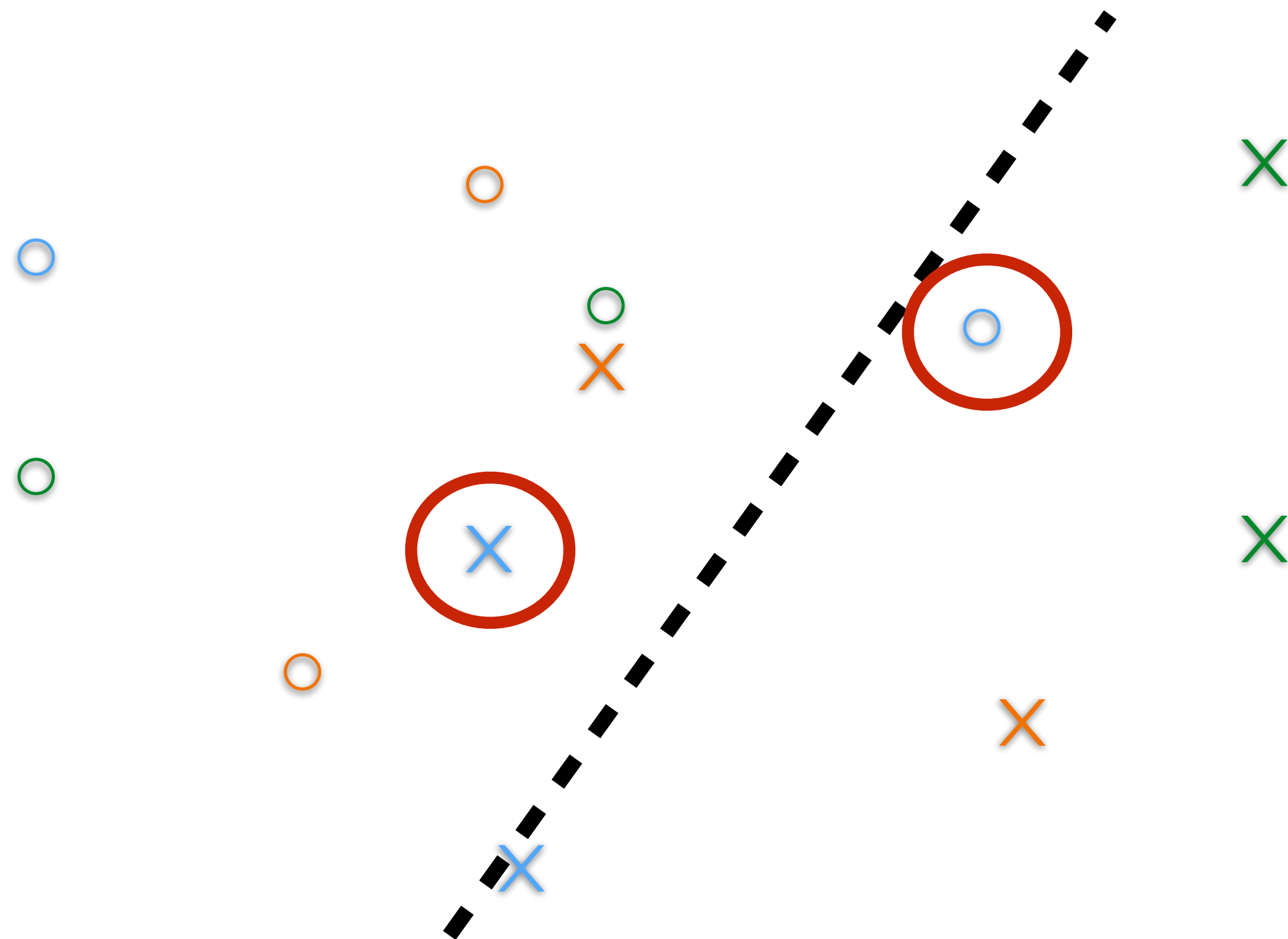
add back orange group: error 2/4

Cross- validation



remove blue group, fit on rest

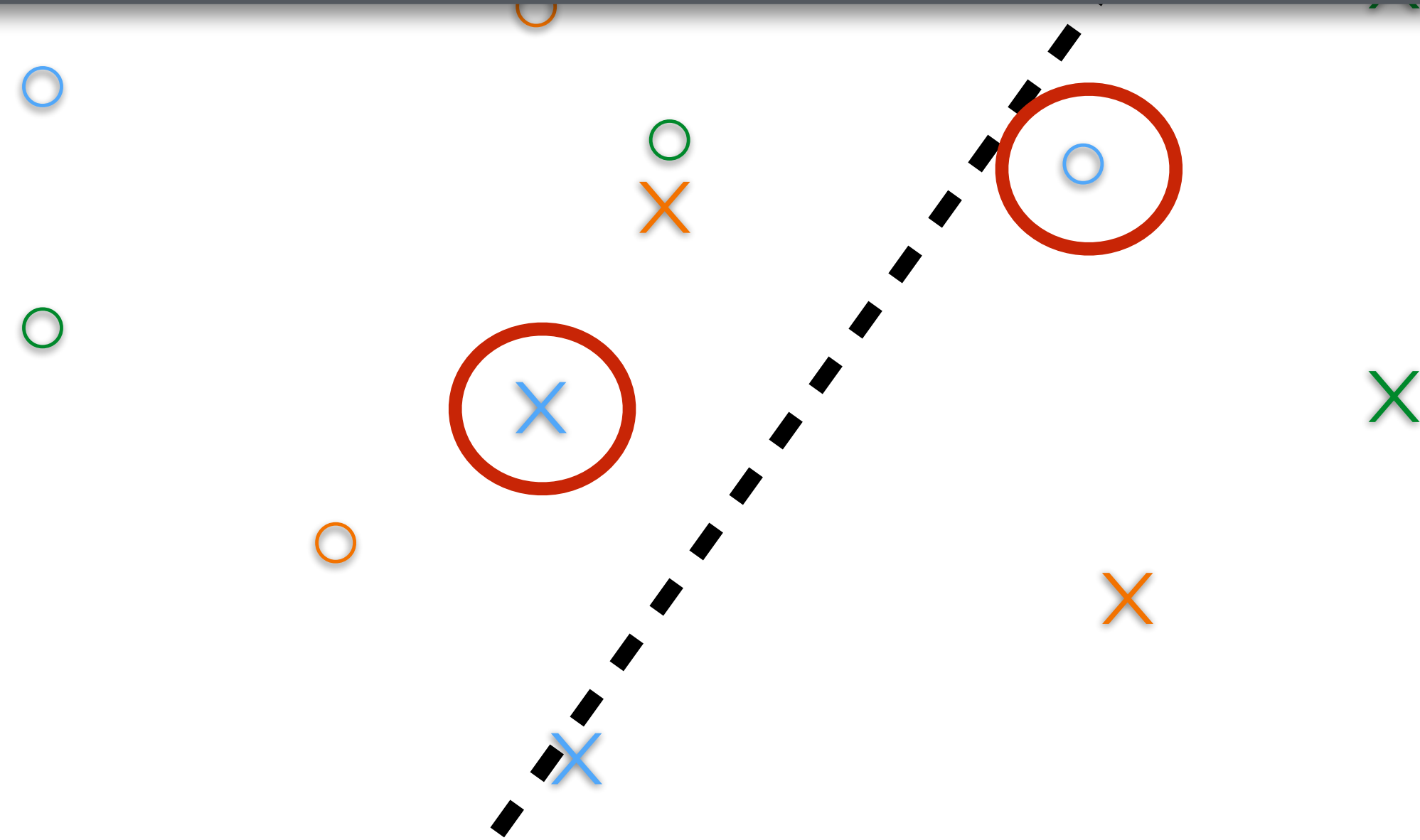
Cross-validation



add back blue group: error 2/4

Cross-validation

Overall: $(1+2+2)/12 = 42\%$ error rate



add back blue group: error 2/4

Model selection with cross- validation

- Split into $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{test}}$
- Split $\mathcal{D}_{\text{train}}$ into folds
- For each model:
 - ▶ for each fold F
 - ▶ train on $\mathcal{D}_{\text{train}} \setminus F$
 - ▶ evaluate on F
 - ▶ average performance over folds — track best model
- Retrain single best model on all folds
- Finally, evaluate resulting trained model on $\mathcal{D}_{\text{test}}$

Model selection with cross- validation

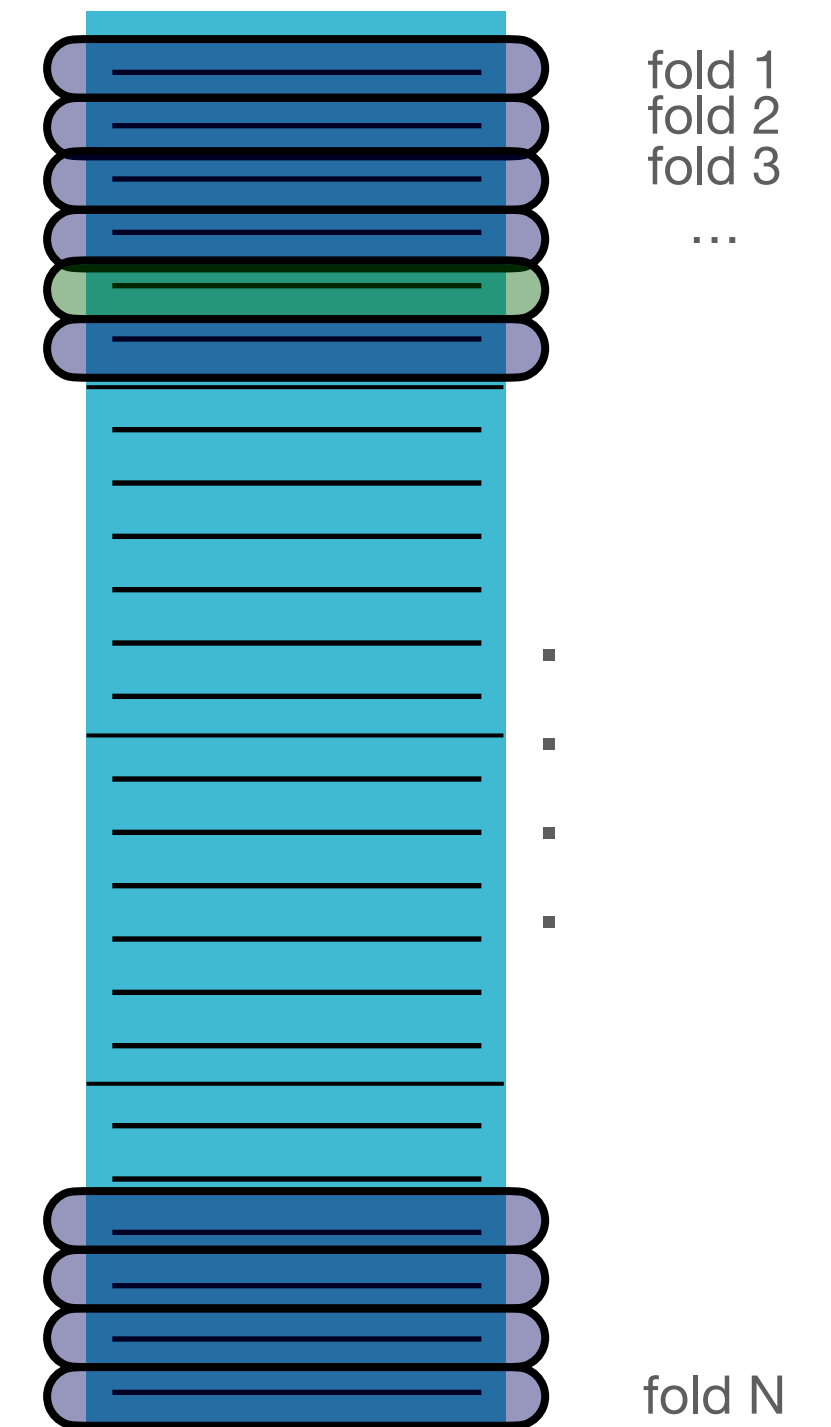
- Split into $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{test}}$
- Split $\mathcal{D}_{\text{train}}$ into folds
- For each model:
 - ▶ for each fold F
 - ▶ train on $\mathcal{D}_{\text{train}} \setminus F$
 - ▶ evaluate on F
 - ▶ average performance over folds — track best model
- Retrain single best model on all folds
- Finally, evaluate resulting trained model on $\mathcal{D}_{\text{test}}$

could also return ***all*** trained models, use them in an ***ensemble*** (e.g., by averaging their predictions)

LOOCV

special case: each
point is its own fold

- Useful special case: *leave-one-out* CV
 - ▶ each point is its own fold: $k = N$
- Keeps training set size as large as possible ($N - 1$)
- Each validation sample is as close as possible to independent
- Expensive — unless there are shortcuts



Learning objectives— kNN and model selection

- How do we train a kNN classifier? How do we predict?
- What is a Voronoi diagram?
- How do k and the distance function influence kNN?
- What is model selection? Why might we want to do it?
- What is a hyperparameter? For example, what are the hyperparameters of kNN?
- How can we do model selection with a train/validation split? How can we do it with cross-validation?

Maximum likelihood



10-701 Introduction to Machine Learning
Geoff Gordon

MOO!



- We've described several ML methods so far by giving *model, objective, optimizer*
- Often three good questions to ask about a new method — maybe not complete, but a good place to start

MOO examples



- Majority
 - ▶ model = constant functions
 - ▶ objective = minimize sum of squared errors
 - ▶ optimizer = exact (compute the mean)

MOO examples



- ID3
 - ▶ model = decision trees
 - ▶ objective = high mutual information, low depth, high number of examples per leaf node
 - ▶ optimizer = greedy top-down induction followed by reduced error pruning

MOO examples



- Gzip:
 - ▶ model: codebook tree + duplicate strings
 - ▶ objective: minimize bit-length
 - ▶ optimizer: [Deflate](#)

Probabilistic machine learning

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- A recipe for making models and objectives
- For model, start from *generative story* of our data
 - ▶ e.g., first step $x^{(i)} \sim N(\mu, I)$ where $\mu \in \mathbb{R}^d$
 - ▶ ... then second step $y^{(i)} \sim N(w^\top x^{(i)} + b, 0.1^2)$
- Each story leads to a different model
 - ▶ maybe instead $x_j^{(i)} \sim \text{Bernoulli}(\pi_j)$ so $x^{(i)} \in \{0,1\}^d$
 - ▶ maybe instead $x^{(i)} \sim$ an arbitrary unknown dist'n
 - ▶ maybe instead $y^{(i)} \sim \text{Bernoulli}(\sigma(w^\top x^{(i)} + b))$
- Each step in the story has *parameters* (e.g., μ, π_j, w, b , the arbitrary distribution of $x^{(i)}$)
 - ▶ our goal is to learn (some of) the parameters

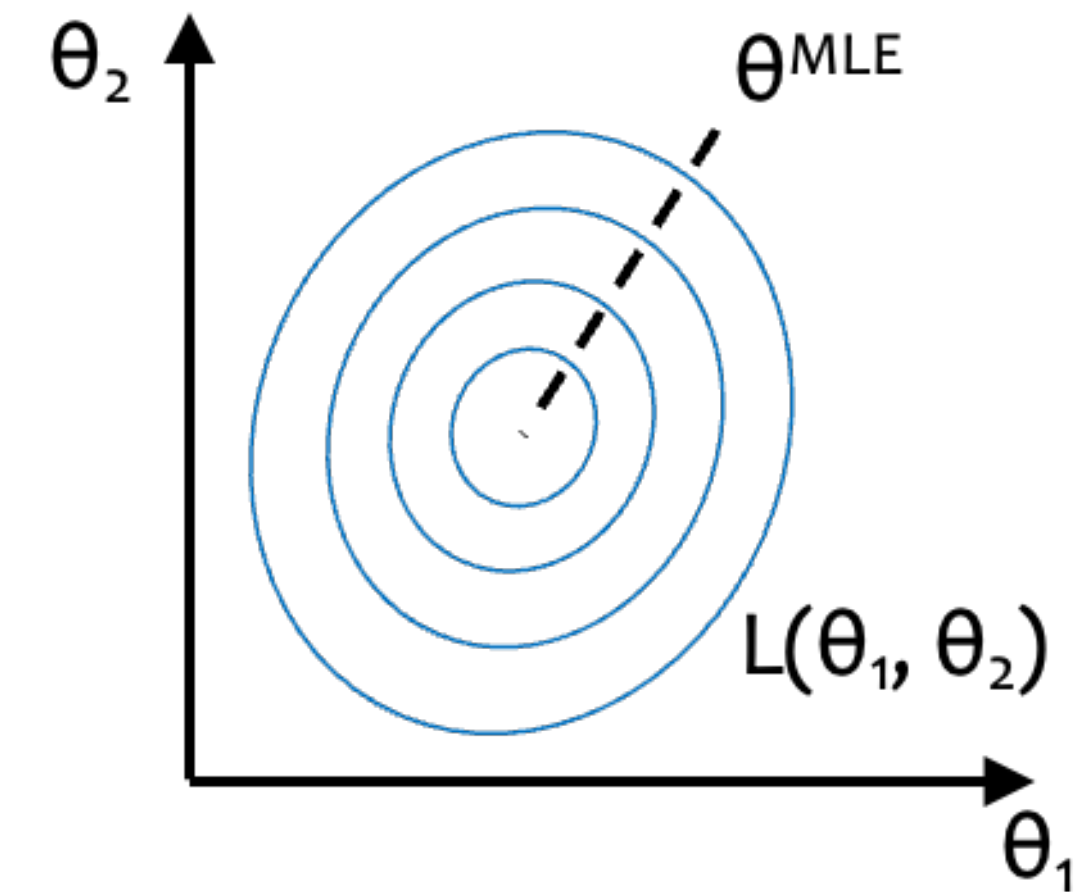
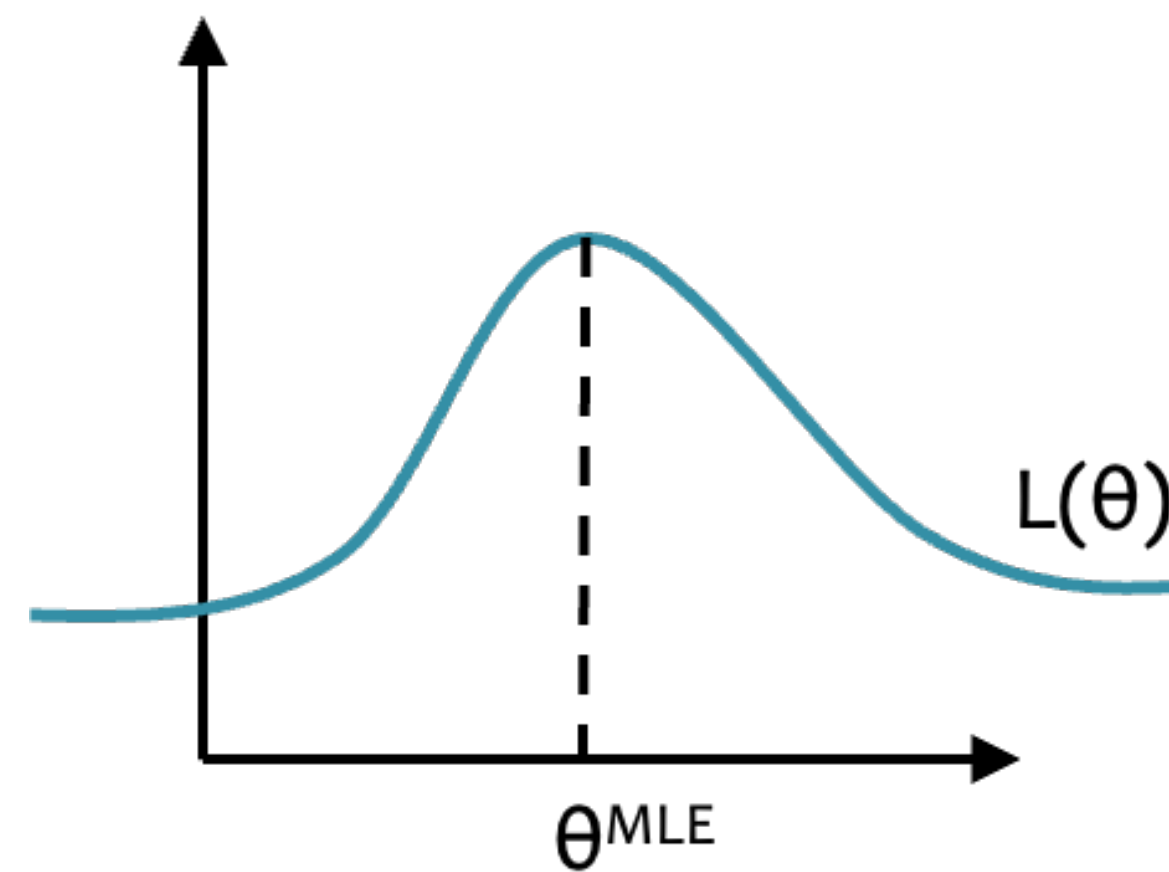
Likelihood

- Objective: collect a training dataset, and use generative story to write out *likelihood*: $P(\text{data} \mid \text{parameters})$, considered as a function of **parameters** (hold data fixed)
 - ▶ or maybe *conditional likelihood*:
 $P(\text{some data} \mid \text{rest of data, parameters})$
 - ▶ useful if it lets us skip learning some of the parameters
- E.g., if $x^{(t)} \sim N(\mu, I)$ (with no $y^{(t)}$),
likelihood $P_{\mu}(x^{(1:t)}) = \prod_{t=1}^N \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}\|x^{(t)} - \mu\|^2}$
- E.g., if $y^{(t)} \sim \text{Bernoulli}(\sigma(w^{\top}x^{(t)} + b))$,
condl. lik. $P_{w,b}(y^{(1:t)} \mid x^{(1:t)}) = \prod_{t=1}^N p_t^{y^{(t)}} (1 - p_t)^{(1-y^{(t)})}$
where $p_t = \sigma(w^{\top}x^{(t)} + b)$

Estimation

Principle of *Maximum Likelihood Estimation*:
Choose the parameters that maximize the
likelihood of the data

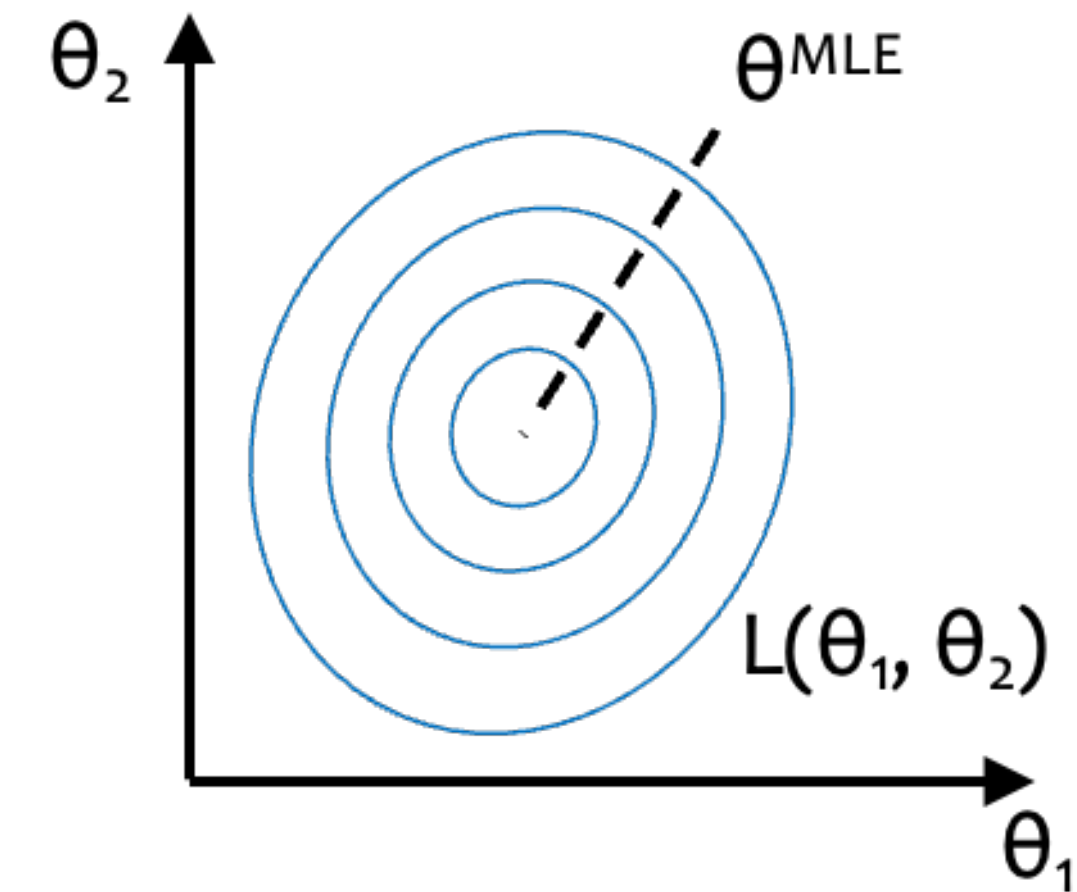
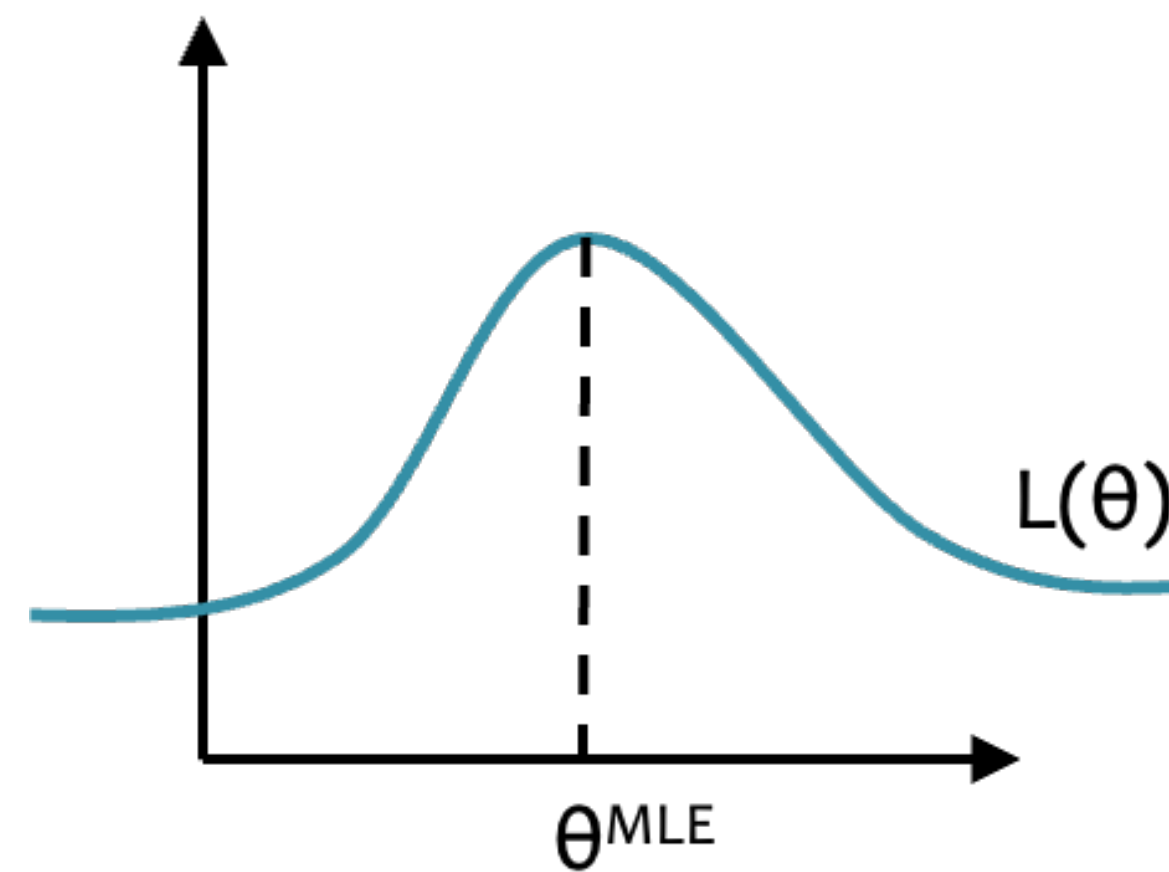
$$\theta^{\text{MLE}} = \arg \max_{\theta} \prod_{t=1}^N P(\mathbf{x}^{(t)} \mid \theta)$$



Estimation

Principle of *Maximum Likelihood Estimation*:
Choose the parameters that maximize the
conditional likelihood of the data

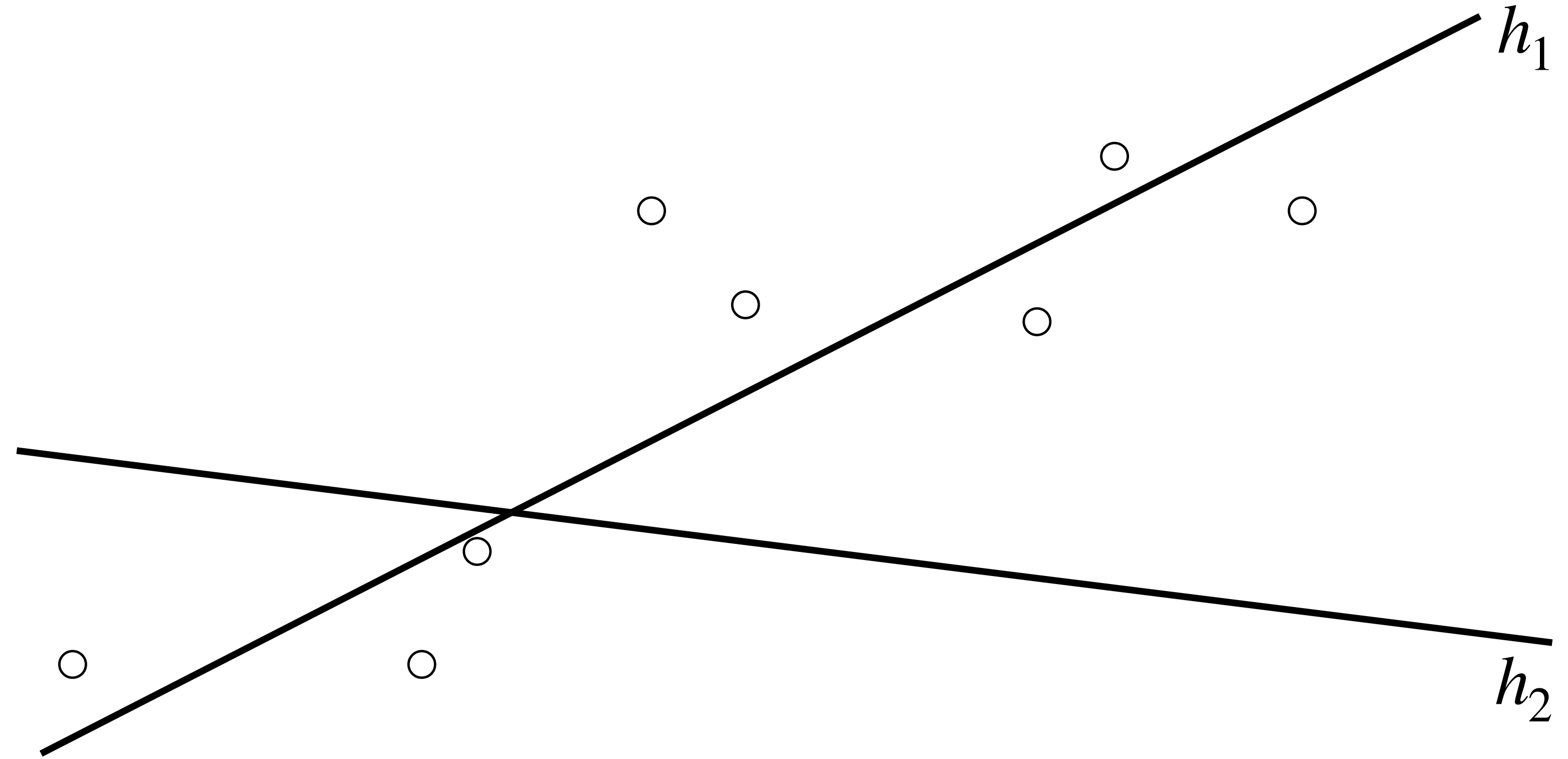
$$\theta^{\text{MLE}} = \arg \max_{\theta} \prod_{t=1}^N P(y^{(t)} \mid \mathbf{x}^{(t)}, \theta)$$



MLE

- Both are often written as ***MLE***
- Choose the model that *best explains the observed data*

Which hypothesis is better?



- $\exists$ only a fixed amount of total probability (sums to one)
- MLE pushes as much probability as possible to things we've observed
 - ▶ ...and steals it from things we didn't observe